

GROUP INTERNSHIP REPORT

EAMS

EAMS Autonomous Rover

Submitted to

Pearl AI Labs (Mulmet Co. Limited)

National ICT Innovation Hub, UICT Nakawa Campus, Kampala, Uganda

Internship Period

22 June 2026 – 7 August 2026

Project Team

1. Nakyeyune Trudy Hope
2. Akankwasa Roberto Sanchez
3. Ampeire Jean Marie Vianney
4. Kyalimpa Mathias
5. Wandwali Chrispus
6. Muliho Denis Wafula
7. Amabe Godwin Pario

Executive Summary

This report documents the design, implementation and testing of the *Engineering Autonomous Mobility System* (EAMS), a low-cost autonomous mobile robot built by a team of seven interns during an eight-week industrial-training placement at Pearl AI Labs at the National ICT Innovation Hub, Nakawa, between 22 June and 7 August 2026. The placement was research-based: the laboratory supplied an inherited four-wheeled chassis and a single goal — a vehicle that can be told to go to a place and drive there on its own, avoiding obstacles on the way — and left the method to the team.

The central engineering problem was to turn that chassis, a marginal low-cost motor driver and a modest compute budget into a platform that could sense its surroundings, decide, and command motion safely. The team addressed it with a simulation-first strategy. A differential-drive digital twin of the vehicle was built in Unity ML-Agents, and a navigation policy was trained on it with Proximal Policy Optimisation using a shaped reward, staged curricula, and deliberate sensor noise and physical randomisation to narrow the gap between simulation and reality. The trained policy was exported to the ONNX format and run on the vehicle's own computer. In parallel, control and telemetry software was written for the physical car: a threaded application serving a browser interface over a WebSocket, in which every command — human or autonomous — passes through one server-side pipeline of five ordered safety guards. Supporting subsystems included a dual-isolated power architecture sized against worst-case motor stall, an Extended Kalman Filter for pose estimation validated in MATLAB, and an early computer-vision prototype.

The platform evolved substantially over the placement. It began on a Raspberry Pi Zero 2 W and moved to an NVIDIA Jetson Orin Nano when on-device policy inference demanded a GPU; the chassis was rewired and rebuilt several times, and the sensor suite was reconciled between the simulator and the real car. Because the report is a single group narrative, it follows the system through these stages rather than tracing seven individual accounts.

By the close of the placement the project had produced a set of concrete, defensible results. The deployed policy reached its target in roughly 79.7 per cent of episodes in the densest obstacle configuration it had trained against. It loads and runs on the Jetson with sub-millisecond inference and has driven the vehicle's real motors under its own control, though on substituted rather than fully live sensor data. The pose filter held position error within a few centimetres in simulation, where unaided dead reckoning would have drifted without bound. The safety layer was verified to hold identically for browser and policy commands. The main remaining limitations are honestly reported: the policy is still fed a zero in place of its own velocity because the vehicle has no wheel

encoders; the dual-battery isolation is designed and built but not yet operating, as the compute module still runs from a mains adapter; the stall-detection threshold has not been validated against real sensor data; and the vision capability remains a prototype. The work delivered is therefore best understood as a working, well-instrumented autonomy testbed together with the scaffolding that will make full on-sensor navigation testable once those gaps are closed.

Acknowledgement

We are grateful to Pearl AI Labs (Mulmet Co. Limited) for hosting this internship and for framing it as genuine research rather than routine placement work. The trust involved in handing a small team the hardware, a single goal and the freedom to work out the method for themselves is the reason the placement taught us as much as it did.

We thank our supervisor at Pearl AI Labs, Mrs. Ainembabazi Jeansbella (Communications Manager), and the field and technical supervisors who guided the project, including Mr. Kwezi, whose well-judged hints and questions redirected our work at the right moments without removing the problem-solving from our hands. We also thank the technical mentors who rotated between teams during build sessions, and the staff of the National ICT Innovation Hub and the Uganda Institute of Information and Communications Technology (UICT), Nakawa, for the facilities and environment that made the work possible.

We acknowledge our respective universities and their industrial-training coordinators for arranging and assessing the placement, and our academic supervisors for their guidance before and during the attachment. Finally, we thank one another: EAMS was built across software, electrical and mechanical disciplines by people who had trained in only one of them each, and it exists because the team was willing to explain its own work and learn everyone else's.

Abstract

EAMS (Engineering Autonomous Mobility System) is a four-wheeled, skid-steer differential-drive robot that converts an inherited low-cost chassis into an autonomous platform governed by edge-deployed artificial intelligence. A navigation policy was trained by Proximal Policy Optimisation within a Unity ML-Agents digital twin of the vehicle, across a graduated obstacle curriculum, and hardened for transfer through domain randomisation and sensor-noise injection. The action space was defined as throttle and steer rather than independent wheel commands, which made turning discoverable early in training and matches a real differential-drive controller; the reward was shaped, with per-step and proximity penalties added to close degenerate solutions. The policy was exported to ONNX and executed on an NVIDIA Jetson Orin Nano, to which the control stack had been ported from an initial Raspberry Pi Zero 2 W prototype through a single narrow hardware-abstraction interface, so that the safety layer transferred unchanged. Because two motors share each channel of an L298N driver that has no thermal protection of its own, thermal safety is enforced entirely in software, through five ordered guards — deadman, stall latch, slew limit, zero-cross coast and a 55 per cent duty cap — placed server-side so that no command source, human or learned, can bypass them. Perception uses paired ultrasonic range-finders, a six-element infrared array and an inertial measurement unit; pose estimation by an Extended Kalman Filter was validated in MATLAB, bounding position error to a few centimetres where dead reckoning diverges. At the deployed checkpoint the policy reached its target in roughly 79.7 per cent of episodes in the densest arena. The policy runs on-device and has driven the real motors under its own control on substituted sensor data; full navigation on live sensors, requiring odometry the vehicle currently lacks, remains the principal open item.

Contents

Executive Summary i

Acknowledgement iii

Abstract iv

List of Acronyms xi

1 Introduction 1

1.1 Background to the Internship . . . 1
1.2 Background to EAMS . . . 1
1.3 Problem Statement . . . 1
1.4 Project Objectives . . . 2
1.5 Report Organisation . . . 2

2 Host Organisation and Project Context 4

2.1 Pearl AI Labs . . . 4
2.2 Relevance of EAMS to the Host Environment . . . 5
2.3 Internship Setting . . . 5
2.4 Project Starting Point . . . 5

3 Requirements, Objectives and Scope 7

3.1 Functional Requirements . . . 7
3.2 Non-Functional Requirements . . . 7
3.3 Scope . . . 8
3.3.1 Included . . . 8
3.3.2 Excluded or Deferred . . . 8
3.4 Engineering Constraints . . . 9
3.5 Success Criteria . . . 9

4 System Architecture and Methodology 10

4.1 System Overview . . . 10
4.2 Hardware Architecture . . . 11
4.3 Software Architecture . . . 11
4.4 Simulation-to-Real Workflow . . . 12
4.5 Development Methodology . . . 13
4.6 System Evolution . . . 13

5 Hardware, Power and Electronics **15**
5.1 Chassis and Drive System . . . 15
5.2 Compute Platform . . . 15
5.3 Motor Driver . . . 16
5.4 Power Architecture . . . 16
5.5 Battery Management and Protection . . . 17
5.6 Circuit Design and Simulation . . . 18
5.7 Hardware Limitations . . . 18

6 Sensing and State Estimation **20**
6.1 Sensor Requirements . . . 20
6.2 Sensor Suite . . . 20
6.3 Sensor Placement and Calibration . . . 21
6.4 Sensor Fusion . . . 22
6.5 Extended Kalman Filter . . . 22
6.6 Validation . . . 24

7 Simulation and Reinforcement Learning **26**
7.1 Motivation for Simulation . . . 26
7.2 Unity ML-Agents Environment . . . 26
7.3 Observation Space . . . 27
7.4 Action Space . . . 27
7.5 Reward Engineering . . . 28
7.6 Curriculum Learning . . . 29
7.7 Domain Randomisation . . . 29
7.8 PPO Training . . . 30
7.9 Policy Export and Deployment . . . 31
7.10 Sim-to-Real Considerations . . . 31

8 Embedded Software and Safety Architecture **33**
8.1 Embedded Software Stack . . . 33
8.2 Control and Telemetry Architecture . . . 33
8.3 Motor Control . . . 34
8.4 ONNX Policy Runner . . . 35
8.5 Safety Layer . . . 36
8.6 Risk Assessment . . . 37
8.7 Deployment on Jetson Orin Nano . . . 38

9 Computer Vision **39**
9.1 Role of Vision in EAMS . . . 39
9.2 Camera and Edge Deployment . . . 39
9.3 Vision Pipeline . . . 40
9.4 Prototype Results . . . 40
9.5 Relationship to Autonomous Navigation . . . 40
9.6 Future Vision Roadmap . . . 41

10 Implementation, Integration and Testing **42**
10.1 Development Phases . . . 42
10.2 Hardware Assembly and Bring-up . . . 42
10.3 Raspberry Pi Pre-Validation . . . 43
10.4 Migration to Jetson Orin Nano . . . 43
10.5 Integration Testing . . . 44
10.6 Faults and Resolutions . . . 45
10.7 Demonstrations . . . 46

11 Results and Discussion **47**
11.1 Achievement of Objectives . . . 47
11.2 Electronics Results . . . 47
11.3 Sensing and State Estimation Results . . . 48
11.4 Reinforcement Learning Results . . . 48
11.5 Embedded Software and Safety Results . . . 49
11.6 Computer Vision Results . . . 49
11.7 Overall System Performance . . . 49
11.8 Limitations . . . 50

12 Group Internship Experience **51**
12.1 Working Environment . . . 51
12.2 Technical Skills Developed . . . 51
12.3 Engineering and Professional Skills . . . 52
12.4 Challenges Encountered as a Team . . . 52
12.5 Lessons Learned . . . 52
12.6 Relationship Between Learning and EAMS . . . 53

13 Conclusion and Recommendations **54**
13.1 Conclusion . . . 54
13.2 Recommendations to Pearl AI Labs . . . 54
13.3 Future Work . . . 55
13.4 Final Remarks . . . 55

A Project Timeline and Milestones **56**

B Team and Working Model **58**

B.1 Project Team 58

B.2 Working Model 58

B.3 Collaborative Workstreams 59

C Selected Code and Configuration **60**

C.1 Safety Constants and Pin Map 60

C.2 PPO Training Configuration 60

C.3 PWM Initialisation Order (a safety-relevant fix) 61

D Test Records **63**

E Project Photographs **65**

List of Figures

4.1 From simulation to the robot 10
4.2 The on-board application 12

5.1 Power and drive architecture 16
5.2 Circuit simulation 18

6.1 Pose EKF block diagram 24
6.2 EKF validation in MATLAB 25

7.1 PPO training reward 30

9.1 Vision prototype 40

E.1 The assembled EAMS differential-drive vehicle. 65
E.2 The vehicle through the build: rewired on arrival and moved from breadboard to soldered stripboard, with the computing board changed from the Raspberry Pi to the Jetson Orin Nano. 65
E.3 The browser control interface during integration testing, showing live telemetry and the status of each safety guard. 66

List of Tables

4.1 Control and data rates on the physical vehicle. 11

6.1 EAMS sensor suite as fitted. 20

8.1 Sources of the eighteen observation values on the vehicle. 36

10.1 Development phase gates. 42
10.2 Selected faults encountered and their resolution. 45

11.1 Achievement of project objectives. 47
11.2 Principal limitations of the system in its present state. 50

A.1 Weekly milestones over the placement. 56

B.1 EAMS project team. 58

D.1 Test records. 63

List of Acronyms

AI	Artificial Intelligence
BJT	Bipolar Junction Transistor
BMS	Battery Management System
BOM	Bill of Materials
CV	Computer Vision
DMA	Direct Memory Access
EAMS	Engineering Autonomous Mobility System
EKF	Extended Kalman Filter
GPIO	General Purpose Input/Output
I²C	Inter-Integrated Circuit
IMU	Inertial Measurement Unit
IR	Infrared
MJPEG	Motion JPEG
ML	Machine Learning
MOSFET	Metal-Oxide-Semiconductor Field-Effect Transistor
ONNX	Open Neural Network Exchange
PPO	Proximal Policy Optimisation
PWM	Pulse Width Modulation
RL	Reinforcement Learning
ROS	Robot Operating System
SLAM	Simultaneous Localisation and Mapping
ToF	Time of Flight
UICT	Uganda Institute of Information and Communications Technology
USB	Universal Serial Bus
VIO	Visual-Inertial Odometry
VLM	Vision-Language Model

1 Introduction

This chapter establishes the purpose of the report and frames EAMS as a single group engineering project carried out during the internship.

1.1 Background to the Internship

Industrial training places a student in a working engineering environment, where problems are real, constraints come from cost and availability rather than a syllabus, and a design is judged by whether it works on hardware. The seven members of this team undertook their placement at Pearl AI Labs during the 2026 recess term, drawn from software, mechatronic and allied engineering programmes at their respective universities. What made the placement unusually valuable was its form: rather than shadowing existing work or delivering a fixed client brief, the team was assigned a single substantial internal research project and given the freedom to work out how to build it. Because the report that follows is about that project, this section is kept deliberately short; the internship's professional and learning outcomes are returned to in Chapter 12.

1.2 Background to EAMS

EAMS, the Engineering Autonomous Mobility System, is a small four-wheeled differential-drive robot — an RC-car-sized platform — built around an inherited laser-cut chassis. It has no steering linkage and changes direction by driving its left and right wheels at different speeds, a scheme known as skid or differential steering. Its onboard computer is an NVIDIA Jetson Orin Nano, adopted partway through the project in place of an initial Raspberry Pi Zero 2 W. The stated ambition for the platform was simple to express and demanding to achieve: a vehicle that can be told to go to a place and drive there on its own, sensing and avoiding obstacles as it goes. Realising even part of that ambition requires combining sensing, embedded computation, motion control, physics simulation and machine learning on a single low-cost vehicle, and it is that combination, rather than any one subsystem, that defines the project.

1.3 Problem Statement

An autonomous vehicle moving through a cluttered space faces a standing conflict. Reliable obstacle avoidance favours rich perception and planning, both of which cost computation,

while safe operation favours a short and predictable reaction time. On a small platform with limited power and low-cost actuation, the conflict cannot be resolved simply by adding hardware. The engineering problem for EAMS was therefore to convert the available vehicle hardware into a useful autonomous mobile platform — one that senses its environment, makes navigation decisions, and commands motion safely — within tight compute, power and cost budgets, and around a single motor driver that operates close to its limit under load. Three specific difficulties follow from this and recur throughout the report: acquiring the navigation behaviour without training on the fragile physical vehicle; maintaining electrical integrity when the motors draw their largest current at exactly the moments avoidance is most needed; and operating safely within the drive hardware's limits without depending on the good behaviour of a learned controller whose outputs cannot be fully predicted.

1.4 Project Objectives

1. Develop and integrate an autonomous rover platform around the available vehicle hardware, carrying the compute module and sensor suite, with a low-level safety layer independent of the AI stack.
2. Establish a simulation and reinforcement-learning workflow, training a navigation policy in Unity ML-Agents across a curriculum of increasing difficulty and exporting it for embedded deployment.
3. Transfer the trained policy to the physical vehicle and run it on-device.
4. Develop sensing and state-estimation capabilities appropriate to the available hardware, including pose estimation by sensor fusion.
5. Implement embedded control, communications, and safety mechanisms through which every command, human or autonomous, must pass.
6. Design the power and drive electronics against worst-case conditions, and investigate a computer-vision capability that can support future autonomy.
7. Validate the integrated system through staged testing and document its limitations honestly.

1.5 Report Organisation

The report is organised as a single project narrative. Chapter 2 describes the host organisation and the state of the project at the point the team inherited it. Chapter 3 sets out the requirements, scope and success criteria, and Chapter 4 the overall system

architecture and development method. Chapters 5 to 9 treat the subsystems in turn: hardware and power electronics; sensing and state estimation; simulation and reinforcement learning; embedded software and safety; and computer vision. Chapter 10 follows the project through implementation, integration and staged testing; Chapter 11 reports and discusses the measured outcomes; Chapter 12 reflects on the collective internship experience; and Chapter 13 concludes with recommendations and future work. Detailed records — the timeline, the team working model, selected code and configuration, test matrices and photographs — are placed in the appendices and referenced from the main text.

2 Host Organisation and Project Context

This chapter introduces Pearl AI Labs, explains how a robotics project fits its mission, describes the working environment of the placement, and records the state of the project at the point the team took it on.

2.1 Pearl AI Labs

Pearl AI Labs is a technology research and training organisation operated by Mulmet Co. Limited, based in the IoT Room at the National ICT Innovation Hub on the UICT Nakawa campus in Kampala [Pearl AI Labs, 2026]. It presents its mission as treating intelligence as essential infrastructure for an intelligent economy, and pursues that mission along two connected lines: foundational research and the practical development of AI systems, alongside intensive, hands-on training in artificial intelligence, data science, robotics and embedded systems for university students on placement and recent graduates.

The laboratory's research is organised around distinct tracks. Its foundational-AI work includes AIHub, a platform intended to give researchers, developers and businesses in Uganda access to state-of-the-art models and tools, and Minuteman, a speech-to-text engine built for African languages and accents. Its embodied-AI track conducts hands-on research into robotic systems, training machines to perform physical tasks through machine learning and building toward industrial and educational robotics for the regional market. Around this research sit three service pillars: an innovation-consultancy arm that builds bespoke AI solutions, autonomous agents and robotics integrations; a products division offering AI training programmes, software and SaaS applications, intelligence dashboards and institutional model licensing; and a 3D-printing service providing rapid prototyping and custom parts that bridge software and physical hardware. A recurring theme across these activities is "sovereign AI" — the aim of letting Ugandan institutions deploy capable models on their own terms, on-premise or via private clouds.

The placement was supervised by Mrs. Ainembabazi Jeansbella, Communications Manager at Pearl AI Labs, with technical personnel and field supervisors brought in to guide the engineering work.

2.2 Relevance of EAMS to the Host Environment

EAMS sits squarely within the laboratory's embodied-AI track. The organisation's stated interest in physical intelligence — sensor fusion, robotic control, and educational robotics platforms suited to the local market — maps directly onto a low-cost autonomous rover that must sense, decide and act on inexpensive, widely available hardware. Two features of the host environment shaped the project in ways worth noting. First, because the facility is a training centre rather than a commercial firm, the placement was not built around a client contract with external deadlines and acceptance criteria; trainees were instead assigned a substantial internal engineering project. This allowed far wider exposure than a commercial attachment would usually permit — one project spanning simulation, electronics, sensor fusion, embedded deployment and vision — but it also meant the team's own internal review cycle had to supply the discipline that a client would otherwise impose. Second, the laboratory's "sovereign AI" and low-resource orientation is consistent with the project's central engineering stance: build the least-capable system that meets the requirement using the hardware actually available, and then show that it does, rather than reaching for the most capable technology on offer.

2.3 Internship Setting

The placement ran for eight weeks, with a working day from roughly 08:30 to 16:00, Monday to Friday. Each day opened with a stand-up meeting in which every team reported the previous day's work, the plan for the day, and any blockers, and closed with each intern updating a project log through a version-control commit or a lab-notebook entry. The morning session was instructor-led technical training during the first three weeks and became dedicated project-build time from the fourth week onward — a structure reflected in the work itself, with the early weeks spent learning tools and building small experiments and the later weeks spent building the system. Interns worked in small teams with a rotating weekly leader, so responsibility for coordinating the work moved around the team rather than resting with one person. Progress was presented for critique at intervals, and the project was demonstrated during a ministerial visit to the laboratory. The daily and weekly logs kept throughout are the reason this report could be assembled accurately rather than from memory; they are summarised in Appendix A.

2.4 Project Starting Point

The team did not begin from a blank sheet. It inherited a physical starting point — a premade four-wheeled laser-cut chassis with four TT gear motors — together with a

research goal and a briefing from the previous project builder. The remainder was open. The assignment was research-based: the laboratory supplied the hardware and periodic guidance, and the interns worked out the approach. Guidance came as well-judged hints and pointers — a suggested video to watch, or a question that redirected an afternoon — rather than as instruction, so a large part of the work was deciding what to build next and why, and teaching oneself whatever that required. The technical work was organised into work packages that recur in the chapters below: reinforcement-learning navigation, sensor fusion and state estimation, embedded control and safety, power and electronics, and an early computer-vision strand. From this starting point the system was built up incrementally, each stage producing something working before the next began, until a policy trained entirely in simulation was driving the physical vehicle's motors.

3 Requirements, Objectives and Scope

This chapter states what EAMS was required to do, the constraints it had to do it within, and the boundary the team drew around the work. It separates what was in scope from what was deliberately excluded or deferred, and sets out the criteria by which the result is judged in Chapter 11.

3.1 Functional Requirements

The system was required to provide, in order of the phases through which it was built:

1. **Manual control with live telemetry.** An operator must be able to drive the vehicle from a browser over a wireless link and see its sensor state live — distances, IR proximity, inertial readings, the duty actually applied to each channel, and the status of every safety guard.
2. **Sensing.** The vehicle must acquire forward and rearward range (ultrasonic), short-range presence around the body (six IR sensors) and its own rotation rate (IMU), and publish these as a single consistent telemetry frame.
3. **Autonomous point-to-point navigation.** A trained policy must be able to drive the vehicle toward a target while avoiding obstacles, replacing the human driver.
4. **On-device inference.** The trained policy must run on the vehicle's own computer, not on a remote server, and issue drive commands in real time.
5. **Unified command path.** Every command, whether from the browser or the policy, must pass through one server-side safety pipeline, with no route around it.

The longer-term ambition — taking a natural-language instruction and choosing a target from it — was recognised as the project's ultimate goal but was explicitly a later phase and was not undertaken during this placement.

3.2 Non-Functional Requirements

- **Thermal safety.** Because two motors share each L298N channel and the driver has no thermal protection of its own, the control software must keep the driver within its thermal envelope under all commands, including sustained stall.
- **Bounded reaction time.** The path from sensing to actuation must be short and predictable, favouring a reactive controller over one that pauses to plan.

- **Power integrity.** Motor current transients, which peak at stall, must not depress the compute supply rail far enough to reset the computer at the moment inference is most needed.
- **Logic-level compatibility.** Both computing boards present 3.3 V inputs that are not 5 V tolerant, so every sensor line reaching the header must be within that limit.
- **Reproducibility and deployability.** The build must be reproducible from documentation, run as a system service from power-on, and offer a hardware-free dry-run mode so software can be tested without the vehicle.
- **Low cost and availability.** The design must use inexpensive, widely available parts, consistent with the platform's purpose as a low-cost robotics testbed.

3.3 Scope

3.3.1 Included

Within scope were mechanical and electrical integration; power-system design; sensor selection and interfacing; the Unity simulation and policy training; policy export and on-device deployment; the embedded control and safety software; pose estimation by sensor fusion; and an early computer-vision prototype.

3.3.2 Excluded or Deferred

Four capabilities were deliberately left out, each for a stated engineering reason rather than for lack of time.

- **Simultaneous localisation and mapping (SLAM)** was excluded because the navigation model is reactive: it avoids obstacles within a bounded reaction time and does not need to place itself on a global map. Mapping would consume compute continuously for a capability the mission does not require.
- **Visual-inertial odometry and stereo depth** were excluded because the ultrasonic and infrared sensors cover their function for present purposes.
- **ROS 2 and Isaac ROS** were excluded because the added abstraction was not justified at the current size of the software stack.
- **The natural-language instruction layer** (choosing a target from a spoken or typed command) was recognised as the project's ultimate goal but not started; it depends on autonomous navigation being solved first.

Path planning, general odometry, data-logging databases and user accounts were also treated as non-goals for this stage.

3.4 Engineering Constraints

Several constraints were fixed and shaped nearly every downstream decision:

- **One L298N driver**, with two TT motors paralleled per channel — a hard constraint that doubles the current per channel and forces thermal protection into software.
- **3.3 V, non-tolerant logic** on the compute header, which dictated powering the sensors from the board's own 3.3 V rail.
- **Hardware PWM only on specific header pins**, which had to be discovered and verified on the actual board rather than trusted from published pinouts.
- **No wheel encoders or odometry**, so the vehicle cannot directly measure its own speed — a constraint that later became the principal sim-to-real gap.
- **An eight-week window** and **a team learning robotics as it built**, which made an incremental, working-at-each-step approach essential.

3.5 Success Criteria

The work is judged against the following, revisited in Chapter 11:

- A navigation policy trained in simulation to a reliable target-reaching rate, reported as a success rate rather than by reward, whose scale is arbitrary.
- That policy running on-device and commanding the motors through the unchanged safety layer.
- The safety guards verified on hardware: the deadman stopping the wheels within its timeout, the stall latch behaving correctly, and the duty cap holding.
- Manual driving working with live, consistent telemetry.
- Documentation sufficient for a later team to continue the work.

Where a criterion was only partially met by the end of the placement, Chapter 11 says so explicitly rather than reporting it as complete.

4 System Architecture and Methodology

This chapter gives the shape of the whole system — how the hardware, the onboard software, and the off-vehicle simulation relate — and the working method by which it was built. Subsequent chapters treat each subsystem in depth; the purpose here is to show how the pieces fit before they are examined separately.

4.1 System Overview

EAMS runs as a chain from simulation to actuation. Training happens off the vehicle in Unity ML-Agents. The trained policy is exported as a single ONNX file and copied to the Jetson; the Unity project does not exist on the vehicle, and the vehicle software knows nothing about how the policy was produced. On the vehicle, a sensor layer samples the physical sensors into a shared state block; a policy runner reads that state, assembles an observation vector, runs inference and issues a drive command; and a motor layer applies the safety limits and writes PWM and direction signals to the L298N. Telemetry is published to a browser client over a WebSocket, and an operator can take control through the same interface the policy uses. Two structural points hold throughout the report: the ONNX file is the only thing that crosses from simulation to hardware, and the policy and the browser client are peers — both produce the same kind of command and both pass through the same safety layer, so the policy has no privileged path to the motors.

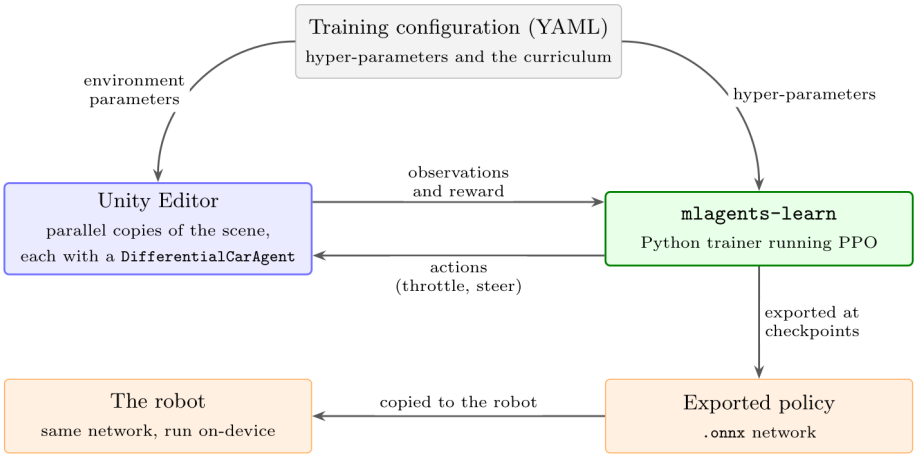


Figure 4.1: From simulation to the robot: training in Unity, export to ONNX, policy on the vehicle.

The system's loops run at fixed, separated rates, summarised in Table 4.1.

Table 4.1: Control and data rates on the physical vehicle.

Loop	Rate	Notes
Motor safety loop	50 Hz	The only code that writes to the motor pins
Sensor sampling	20 Hz	Ultrasonic, infrared and IMU
Policy inference	20 Hz	Matched to the command rate
Telemetry and commands	20 Hz	Over one WebSocket
Motor PWM carrier	2 kHz	Hardware switching frequency

4.2 Hardware Architecture

The vehicle is a four-wheeled, skid-steer differential-drive platform on a two-deck laser-cut chassis: the lower deck carries the battery pack and the four TT gear motors, the upper deck the motor driver and the compute module. All four wheels are driven with no steering linkage, so the vehicle turns by driving its two sides at different speeds. A single L298N dual H-bridge drives all four motors, with the left pair paralleled on one channel and the right pair on the other. The Jetson Orin Nano is the onboard computer; it was specified for the vision work rather than for the navigation policy, which a smaller computer could have run. Perception is provided by two US-100 ultrasonic modules (front and rear), six infrared proximity modules around the body, an MPU6050 inertial measurement unit on I^2C, and a USB webcam for operator video. The full electrical design, the power topology and the pin assignment are the subject of Chapter 5.

4.3 Software Architecture

The onboard software is a single Python application built on Flask with WebSocket support, running as a system service and organised into modules with clearly separated responsibilities: a server that owns the interface and control arbitration, a motor module that runs the safety loop, a sensor module that samples and converts, a policy module that runs inference, a hardware module that provides GPIO, PWM and I^2C access, and a configuration module that holds every pin, constant and limit.

Four threads run concurrently and communicate only through a shared state block protected by a lock, rather than by calling one another directly. The web-and-socket thread serves the interface and owns the WebSocket. The sensor thread samples at 20 Hz into shared state. The motor thread runs the 50 Hz safety loop and is the only code that writes to the motor pins. The policy runner performs ONNX inference at 20 Hz and issues its commands through the same interface the browser uses. The division is deliberate: sensor reads block — an ultrasonic measurement waits for its echo — and a blocking read

must never be able to delay a motor safety decision, so the motor loop's timing is kept independent of everything else.

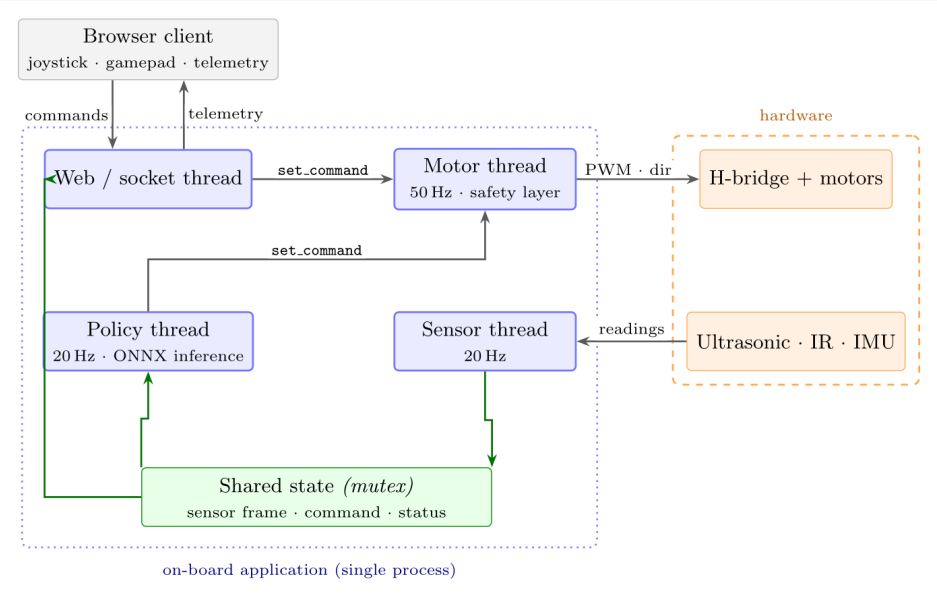


Figure 4.2: The on-board application: its threads, shared state and safety layer.

A single rule holds the safety architecture together: one module, and only one, touches the motor pins, and every command reaches those pins through the same pipeline inside it. All hardware access, in turn, passes through one narrow class, which is what made the later change of computing board tractable (Section 4.6) and what provides the dry-run mode in which the whole server runs on a laptop with no hardware attached — where most logic faults were found.

4.4 Simulation-to-Real Workflow

The project rests on a simulation-first workflow, and on one principle that unifies its simulators: a single ground-truth vehicle state, observed by every sensor. Sensors never invent independent values — accelerate and the modelled speed, rotation and range readings all move together — so the software stack can be developed and validated before the physical sensors are wired. Three kinds of simulation supported this. The Unity ML-Agents environment (Chapter 7) is where the policy is trained. Two Python simulators mirror the real car's telemetry and its WebSocket architecture, so a controller can drive a simulated rover through the same interface as the real one. And within the control software itself, dummy sensor and dummy motor modes — exposed through an "Autonomy Lab" panel — allow the full autonomous pipeline to be exercised on the real Jetson without the wheels turning.

Making the trained policy behave on the vehicle as it did in simulation required

matching the two environments exactly: the observation vector is assembled in the same order and with the same normalisation constants the Unity agent used, the mixing of throttle and steer into wheel commands is a line-for-line translation of the training code, and even the motor-smoothing that Unity applies is reimplemented on the vehicle, because two of the policy's observations are its own recent smoothed commands fed back to it. A mismatch in any of these would not raise an error — the network would simply receive inputs subtly unlike those it trained on and behave worse for reasons hard to diagnose. This is treated in detail in Chapter 8.

4.5 Development Methodology

The project ran on an iterative process rather than a fixed plan. The team met each morning to report progress and blockers, which is where inconsistencies between the simulation, electronics and software strands were caught, and outstanding work was tracked on a shared board covering general tasks, electronics, software and machine learning. Responsibility for coordinating the week rotated between team members. At intervals a team member presented completed work for critique and sign-off. Source code was kept in Git and hosted on GitHub, and design decisions were written into the configuration file as comments beside the values they justify, keeping the reasoning next to the thing it explains.

Work moved through explicit phase gates, each of which had to pass before the next began: laptop dry-run, Raspberry Pi bring-up, Jetson migration, wheels-off-the-ground testing, and only then driving on the floor. Chapter 10 follows the project through these gates. The consistent preference throughout was to validate in dry-run, in simulation or with the wheels off the ground before anything drove real motors.

4.6 System Evolution

The platform changed substantially over the eight weeks, and the report treats that evolution as part of the engineering rather than as noise. The compute module moved from a Raspberry Pi Zero 2 W to an NVIDIA Jetson Orin Nano once on-device policy inference demanded a GPU; because all hardware access sat behind one interface, only that backend was rewritten and the entire safety layer transferred unchanged. The electronics were rebuilt from breadboard onto soldered stripboard, the pin map was revised several times as the true PWM-capable header pins were established, and the sensor supply was moved from a 5 V rail to the board's 3.3 V rail once the logic-level mismatch was found. Late in the placement the inertial sensor was brought up and the stall guard re-enabled, and the previously separate rover, simulation and training repositories were consolidated

into one. Each of these steps is recorded with its reasoning in the chapters and appendices that follow.

5 Hardware, Power and Electronics

This chapter describes the physical vehicle and its electrical design: the chassis and drive, the compute platform, the single motor driver that constrains the whole project, the power architecture, the circuit simulation carried out before anything was built, and the hardware limitations carried into the results.

5.1 Chassis and Drive System

The vehicle is built on an inherited two-deck laser-cut chassis. The lower deck carries the battery pack and the four TT gear motors; the upper deck carries the motor driver and the compute module. All four wheels are driven and there is no steering linkage, so the vehicle turns by driving its two sides at different speeds — skid, or differential, steering. The four TT gear motors operate over a 3–12 V range and are arranged as two pairs: the left pair and the right pair, each pair driven together.

Skid steering has two consequences that recur later in the report. Because turning drags the tyres sideways rather than rolling them, the relationship between wheel rotation and vehicle motion is only approximate, which matters for pose estimation in Chapter 6. And because the tyres are scrubbed as well as rolled, current rises during a turn, which matters for the thermal budget of the driver. An enclosure for the electronics was modelled in CAD during the placement.

5.2 Compute Platform

The onboard computer is an NVIDIA Jetson Orin Nano, which runs the control software, the sensor layer, the 50 Hz safety loop, policy inference and the browser interface. It was specified for the vision work rather than for the navigation policy, which a smaller computer could have run. All pin references use Jetson.GPIO BOARD numbering — physical positions on the 40-pin header rather than the processor's own GPIO numbers.

Two other processors were used during development and neither is on the vehicle now. The Raspberry Pi Zero 2 W was the pre-validation platform, on which the motor control, sensor handling and safety layer were proven before the Jetson was available; when the software moved across, only the hardware-access layer was replaced. Separately, an STM32 ran a parallel hardware-validation track carrying the same sensor suite, with a Raspberry Pi logging over a serial link. A microcontroller with hardware timers measures an ultrasonic echo pulse far more precisely than a Linux system running a Python loop, so

this track let sensor and wiring faults be separated from operating-system and application faults. Bring-up on it was staged one subsystem at a time — power rail, one IR sensor, then six, the ultrasonic pair under sequential triggering, the IMU, motor PWM, then full integration.

5.3 Motor Driver

A single L298N dual H-bridge drives all four motors, with two motors paralleled on each channel: the left pair on channel A and the right pair on channel B. Each channel takes one PWM enable input and two direction inputs. Using one L298N was a fixed constraint of the available hardware, and paralleling two motors per channel doubles the current each channel must carry, which is the source of the project's central electrical problem.

The L298N is a bipolar-transistor bridge: it drops roughly one to two volts across each conducting device more or less regardless of current, so the power dissipated inside the driver is that drop multiplied by the load current and rises linearly with it. In practice the device is limited by the heat its package can reject rather than by the current its transistors can carry. A MOSFET driver would switch through a small resistance instead, with losses one to two orders of magnitude lower at these currents, and modern integrated parts add current limiting and thermal shutdown — neither of which the L298N provides. On technical grounds the MOSFET part is the better choice; the L298N persists in low-cost robotics through availability rather than merit, and it was the hardware to hand. The consequence, developed in Chapter 8, is that thermal protection has to be supplied in software.

5.4 Power Architecture

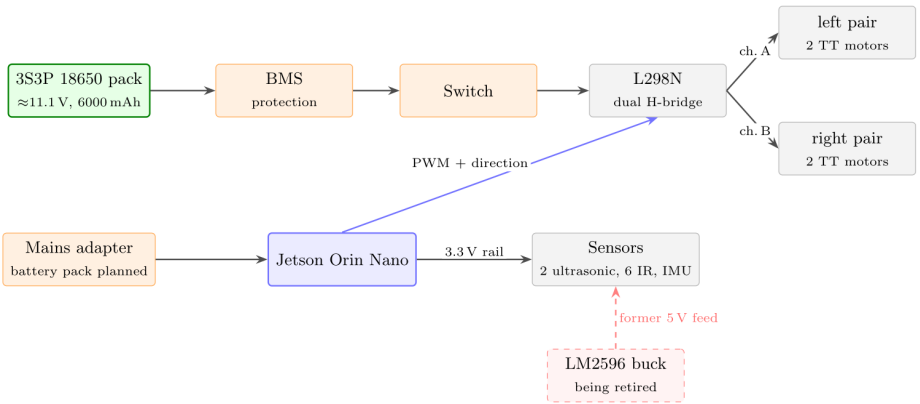


Figure 5.1: Power and drive architecture of the vehicle.

The power design separates the motor and compute domains. A 3S3P pack of 18650 lithium-ion cells — about 11.1 V nominal, 12.6 V full — supplies the L298N motor rail directly. A second, smaller 3S2P pack is intended to supply the compute module, so that motor-current transients cannot reach the Jetson's supply. The two domains are designed to share only a common ground, joined at the pack negative in a star arrangement so that motor return current is kept out of the logic ground, where the shared-conductor resistance would otherwise corrupt sensor readings. The L298N's onboard 5 V regulator jumper is removed and its regulator left unused. An LM2596 step-down converter was originally fitted to supply the sensors at 5 V.

The reason the two supplies must be isolated rather than merely decoupled is a difference in load character. A motor draws a large, abrupt current, greatest at stall, while a processor draws a steady current but resets if its rail sags. On a shared rail a stall transient reaches the processor directly, and the resulting failure is correlated with demanding operation — exactly obstacle avoidance — rather than being random. Decoupling addresses only the high-frequency part of the disturbance; isolating the supply paths removes the coupling.

Build status. This is a point where the report distinguishes carefully between what was designed and what is operating. Both packs were built, and the 3S3P pack is in service supplying the motors and sensors. The 3S2P compute pack, however, is not yet connected: for the work described here the Jetson ran from a mains adapter. The dual-isolated architecture is therefore designed, analysed and built, but not yet operating, so the vehicle was not fully untethered and the isolation has not been tested under stall.

5.5 Battery Management and Protection

Lithium-ion cells require protection against overcharge, over-discharge and excessive current, and series cells drift apart in state of charge unless balanced. The series-versus-parallel distinction is central to the power design: cells in series add voltage while the current capability stays that of a single cell, whereas cells in parallel add capacity and current at the same voltage. Adding cells in series therefore does not raise the current a pack can deliver — only more parallel strings or higher-rated cells do. Because the vehicle's peak load at stall is several times its average, the pack must be sized against the peak, and it is the peak rather than the endurance requirement that drove the pack configuration. A battery management and protection scheme was designed and simulated for the pack, covering the protection functions above.

5.6 Circuit Design and Simulation

The electronics were designed and simulated in Proteus 8 Professional before anything was committed to hardware, so that rail voltages and behaviour could be checked in simulation rather than discovered on the bench. Three simulations were built: the battery management and protection circuit, the L298N motor drive, and an integrated schematic combining the verified blocks. Because the exact parts were not all present in the simulator's library, a number of component substitutions were made, chosen to preserve the electrical behaviour under test rather than the exact part number. The circuit work was presented to the team in the Week 3 electronics review, where the block designs were checked against one another for agreement on rail voltages before either went to hardware.

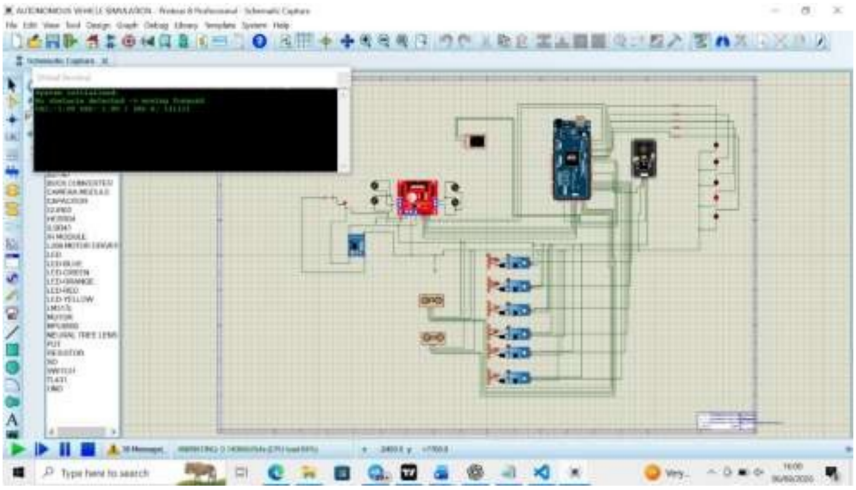


Figure 5.2: Simulation of the drive and sensor circuit before it went to hardware.

5.7 Hardware Limitations

Several hardware limitations are carried forward honestly into the results:

- **The single L298N is marginal at stall.** Two paralleled TT motors present a worst-case stall current close to the channel's practical limit, and the driver has no current limiting or thermal shutdown of its own. This is what forces the software safety layer and is the single most important electrical fact about the vehicle.
- **Real stall current is unmeasured.** The duty cap that protects the driver is set from an estimate; the true per-motor stall current has not yet been measured, and the cap may need to come down once it is.
- **The dual-battery isolation is not yet operating.** As noted above, the compute pack is unconnected and the isolation is untested under stall.

- **Marginal logic levels.** Driving 3.3 V logic into the L298N's inputs is close to the part's threshold on some clones, and is a known source of erratic direction control to suspect before the software.

6 Sensing and State Estimation

This chapter covers how the vehicle perceives its surroundings and estimates its own state: which sensors were chosen and why, how they are interfaced and calibrated, and the Extended Kalman Filter developed to fuse commanded motion with gyroscope measurement. It is careful to distinguish the pose filter, which was designed and validated in MATLAB, from what is currently running on the vehicle.

6.1 Sensor Requirements

Because EAMS navigates reactively and builds no map, it does not need sensors capable of localising against a global frame. It needs obstacle distance in the direction of travel, short-range detection around the vehicle, and a measure of its own rotation for pose estimation and stall detection. Five criteria were applied to each candidate sensor: logic-level compatibility with the 3.3 V header; interface cost in header pins or bus capacity; range overlap, since a sensor duplicating another's coverage adds current for no new information; current draw against the power budget; and simulation feasibility, since any sensor feeding the navigation policy must be modellable in Unity. The last criterion is unusual and worth noting: sensor selection was constrained by the machine-learning approach, not only by electrical and mechanical requirements.

6.2 Sensor Suite

The fitted sensor suite is summarised in Table 6.1.

Table 6.1: EAMS sensor suite as fitted.

Sensor	Model	Qty	Interface and purpose
Ultrasonic	US-100	2	Trigger/echo GPIO; forward and rearward range
IR proximity	Digital module	6	Digital GPIO; presence in six directions
IMU	MPU6050	1	I^2C bus 7, address 0x68; angular rate
Camera	USB webcam	1	USB; operator video and future vision work

The two US-100 ultrasonic modules were chosen over the more common HC-SR04 because they work with 3.3 V logic, which removes a whole class of interface problems rather than solving them with extra components. Two were fitted because the vehicle

reverses, and a rear obstacle stops it as effectively as a front one. The six infrared modules each give a single digital output with a potentiometer setting the trip distance; they report presence, not range, and complement the ultrasonic sensors rather than duplicating them — ultrasonic gives range in two directions, infrared gives presence in six, covering the sides and corners the ultrasonic cones miss. Their order — front-left, front-right, rear-left, rear-right, left, right — matters beyond wiring, because it is also the order the trained policy expects; the pin list and the label list in the configuration are kept parallel so that re-making the harness cannot silently feed the policy correct readings in the wrong positions.

The MPU6050 communicates over I^2C at address 0x68 and produces 16,384 counts per g and 131 counts per degree per second at the settings used. The gyroscope is what matters here: it feeds the pose filter, the policy's yaw-rate observation, and the stall detection of Chapter 8. The accelerometer is reported in telemetry but not used for state estimation, since obtaining position from acceleration requires double integration and diverges too quickly to be useful.

Two candidate sensors were rejected, and the contrast between them is instructive. The HC-SR04 was rejected on logic-level grounds. The VL53L0X time-of-flight sensor was rejected because its useful range was already covered by the infrared and ultrasonic sensors, so it would have added an I^2C device, current draw and wiring for information the vehicle already had — whereas the IMU, which also sits on the bus, was kept precisely because nothing else measures rotation. In both cases the criterion was whether the sensor contributed information no other sensor provided. Wheel encoders were *not* fitted, which is significant and returned to in the pose-estimation discussion below.

6.3 Sensor Placement and Calibration

The ultrasonic modules are mounted front and rear on the lower deck, facing outward along the vehicle axis; the six infrared modules are distributed around the chassis perimeter; and the IMU is mounted on the chassis rather than a moving part, since it must measure the motion of the vehicle body. The sensors take their supply from the board's own 3.3 V rail. This was a deliberate change from an earlier arrangement in which they were powered at 5 V through the step-down converter: at 5 V, eight signal lines — two ultrasonic echoes and six infrared outputs — were presenting 5 V to header pins rated for 3.3 V, out of specification for the board. Because both the US-100 and the infrared modules operate well below 5 V, moving the sensor supply to 3.3 V brings every one of those lines into specification without adding a single level-shifting component.

The sensor thread samples all sensors at 20 Hz into the shared state block. Each ultrasonic reading is a single blocking measurement returning a distance in centimetres, or nothing if the echo does not arrive within its timeout or the value exceeds a validity limit;

a timeout is treated as no reading rather than as maximum range, because a missing echo can mean a clear path, an absorbing surface, or a surface angled so as to reflect the pulse away. The two ultrasonic modules are fired alternately, one per tick, because two units firing together can each receive the other's echo and report a distance that does not exist. The six infrared readings are packed into a single six-bit mask.

Three calibration issues apply. The gyroscope has a zero-rate offset that must be measured at rest and subtracted, or the filter integrates a constant error into heading. Ultrasonic timing depends on air temperature through the speed of sound, a small proportional error not significant at these ranges but worth acknowledging. And the infrared trip distances are set individually by potentiometer, so the six do not trip at identical distances unless set against a reference; since the policy treats all six as equivalent, uneven thresholds make the vehicle more sensitive on some sides than others.

6.4 Sensor Fusion

The simplest way to estimate position is to integrate motion forward from a known start, converting commanded wheel efforts into forward and rotational speed through a differential-drive model. This fails because the estimate has no source of correction: no measurement anywhere in the loop says where the vehicle actually is, so errors accumulate without limit. On this vehicle the errors come from several directions at once. Without encoders, the prediction uses commanded rather than measured wheel motion, so a wheel that slips, stalls, or turns slower than commanded because the battery has discharged goes unnoticed. Skid steering drags the tyres sideways, so the idealised differential-drive equations do not describe the motion exactly. And the effective geometry is uncertain, since the conversion depends on wheel radius and track width, and under skid steering the effective track width is not the mechanical spacing.

Heading error dominates, because a small angular error carried over distance produces a large sideways position error. That is what makes fusion worthwhile: the gyroscope measures rotation directly and independently, with errors arising from a different mechanism than the prediction's, so combining the two gives a heading estimate that is corrected rather than merely accumulated.

6.5 Extended Kalman Filter

An Extended Kalman Filter (EKF) was chosen because the motion model is nonlinear — the relationship between wheel speeds and position involves trigonometric functions of heading — and the EKF handles this by linearising about the current estimate at each step, while remaining cheap enough for a real-time loop and producing a covariance that quantifies its own confidence.

The state carried is the planar pose and its rates,

$$\mathbf{x} = [\, x,\ y,\ \theta,\ v,\ \omega \,]^{\mathsf{T}},$$

where x and y are position, θ is heading, v is forward speed and ω is turn rate. Planar pose is sufficient because the vehicle operates on a flat floor. Carrying v and ω as states rather than treating them purely as inputs is deliberate: it allows the gyroscope to correct the estimated turn rate directly, which then propagates into heading on the following step. If ω were only an input, a gyroscope measurement would have nowhere to act.

The prediction advances the pose over the timestep Δt while holding speed and turn rate:

$$x_k^- = x_{k-1} + v\cos(\theta_{k-1})\,\Delta t, \quad y_k^- = y_{k-1} + v\sin(\theta_{k-1})\,\Delta t, \quad \theta_k^- = \theta_{k-1} + \omega\,\Delta t,$$

with v and ω held. The v and ω used are the *commanded* values, since without encoders that is all the prediction has. The nonlinearity lies in the trigonometric terms, which is why an extended rather than a standard filter is needed; the Jacobian is the identity apart from the two terms that capture how a change in heading moves the predicted position,

$$F_{1,3} = -v\sin(\theta)\,\Delta t, \qquad F_{2,3} = v\cos(\theta)\,\Delta t.$$

Process noise is set as $Q = \mathrm{diag}(\sigma_p^2, \sigma_p^2, \sigma_\theta^2, \sigma_v^2, \sigma_\omega^2)$ with $\sigma_p = 0.01$, $\sigma_\theta = 0.01$, $\sigma_v = 0.05$ and $\sigma_\omega = 0.05$. The speed and turn-rate terms are given five times the standard deviation of the others, which encodes the weakest assumption in the model — that the vehicle is actually moving at the commanded rate — and tells the filter exactly where it should trust the prediction least.

The gyroscope measures turn rate directly, so the measurement model is $H = [\,0\ 0\ 0\ 0\ 1\,]$ with $R = \sigma_{\text{gyro}}^2$ and $\sigma_{\text{gyro}} = 0.03$ rad/s. The filter runs the standard cycle: prediction propagates the state through the motion model and the covariance as $P_k^- = FP_{k-1}F^{\mathsf{T}} + Q$; the update forms the innovation from the gyroscope reading, computes the gain from the current covariances, and corrects the state and covariance. The gain is recomputed each step, so the weighting between prediction and measurement adapts as uncertainty grows and shrinks. The filter was initialised at the origin with covariance $\mathrm{diag}(0.1, 0.1, 0.05, 0.1, 0.1)$ and runs at a 0.05 s timestep, matching the 20 Hz sensor rate on the vehicle.

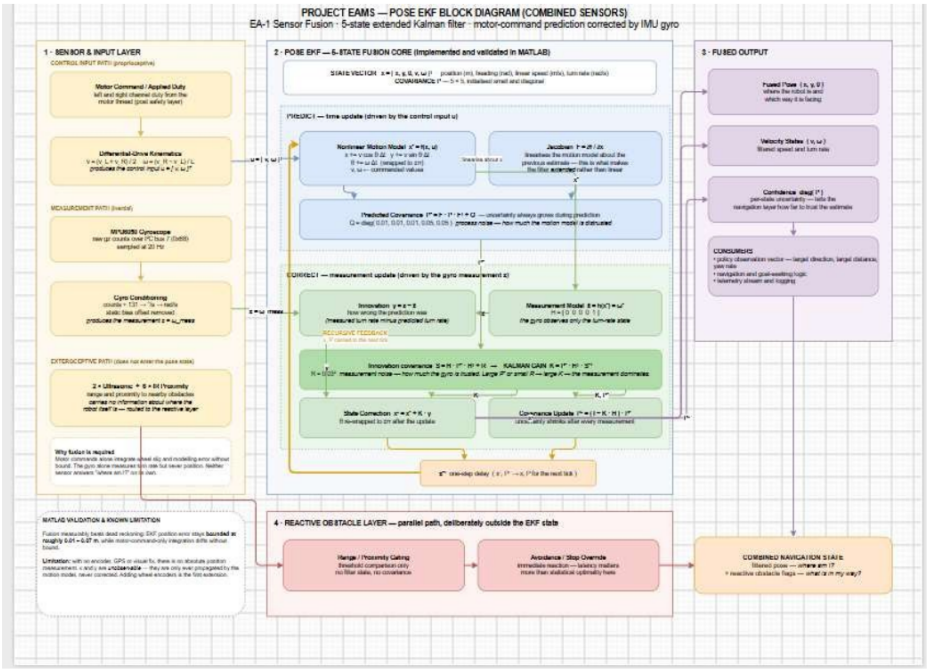


Figure 6.1: Block diagram of the pose Extended Kalman Filter, fusing all sensors.

6.6 Validation

The filter was developed and validated in MATLAB before any attempt to run it on the vehicle, because the estimation problem can be worked on there without real-time scheduling, hardware faults or sensor drivers. Three scripts were built: the pose EKF alone, a reactive obstacle layer alone, and a combined simulation running both in the arrangement they would take on the vehicle. The value of the exercise lies in how the simulation is constructed. It separates three quantities: the commanded speed and turn rate; the true motion, which is the commanded motion plus simulated wheel slip of 0.04 m/s and 0.06 rad/s; and the estimated motion, predicted from the commanded values and corrected by a gyroscope reading taken from the true turn rate plus noise. The prediction is therefore fed commanded motion, so it is wrong in exactly the way the real vehicle's prediction is wrong, while the gyroscope sees the true rotation. The gap between them is what fusion exists to close, and the simulation creates it deliberately so that closing it can be measured against a ground truth that real hardware cannot provide.

Under those conditions the EKF held position error bounded within roughly 0.01 to 0.07 m, peaking near 0.07 m and returning to about 0.02 m by the end of the run, and — the result that matters — fluctuating without trending upward. Unaided dead reckoning under the same slip would accumulate error without bound. The combined script also exposed a limitation honestly: in one configuration the reactive obstacle layer failed to reach the goal, becoming trapped in a local minimum of the kind that defeats purely reactive navigation. That the learned policy holds no explicit map does not remove this

weakness, and it is reported again in Chapter 11.

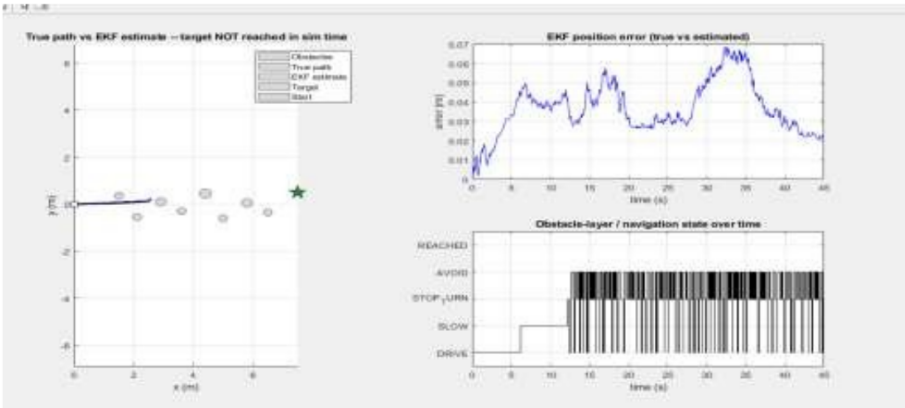


Figure 6.2: EKF validation in MATLAB: true and estimated path, position error and navigation state.

Current use on the vehicle. The pose filter is a validated design, not a deployed subsystem. At the time of writing it runs in MATLAB only; it is not integrated into the vehicle software, and estimated pose currently feeds nothing on the car. Two things stand between the validated filter and live pose on the vehicle: the absence of wheel encoders, which is why the prediction can only use commanded motion, and the integration work of running the filter inside the 20 Hz sensor loop. Fitting encoders is the single change that would most improve the estimate, and it is the same gap that keeps the navigation policy's own velocity observation fed with zeros, discussed in Chapters 7 and 8.

7 Simulation and Reinforcement Learning

This chapter describes how the navigation behaviour was acquired: the Unity digital twin the policy was trained in, the observation and action spaces, the shaped reward and staged curricula, the randomisation that hardened the policy for transfer, the PPO training itself, and how the trained network was exported for the vehicle.

7.1 Motivation for Simulation

Training a navigation policy by reinforcement learning needs a very large number of episodes, and an agent learning to avoid obstacles necessarily begins by colliding with them. Doing that on the physical vehicle would be prohibitively slow and destructive: every episode would need the car returned to a defined start, and the collisions the policy exists to prevent would be inflicted on real hardware thousands of times over. Simulation is the only environment that allows the parallel, repeatable and resettable runs that policy-gradient methods require. It also let the project proceed on two strands at once — the policy could be learning to drive while the vehicle it would eventually drive was still being wired — so training in simulation was a necessity rather than a preference.

7.2 Unity ML-Agents Environment

A digital twin of the vehicle was built in Unity, using the ML-Agents toolkit, which lets a Unity scene serve as a training environment for a learning agent. The simulation was built to resemble the real robot rather than to look realistic: what a navigation policy depends on is the vehicle's turning and acceleration dynamics, actuator lag, friction and sensor noise, not how the scene is rendered.

The most consequential physics decision was to stop moving the car directly. An early implementation translated and rotated the transform, which was simple but produced motion that did not resemble a real vehicle. This was replaced by applying a forward force at each of two points positioned where the left and right wheels meet the ground, and letting the physics engine resolve the translation and rotation. Because the two anchors are offset across the car's axis, unequal left and right forces produce a turning moment about the centre, which is exactly what makes a differential-drive vehicle behave

as one: equal commands drive it straight, opposite commands spin it in place. The forces are applied as accelerations so that the result does not depend on the randomised mass, with a drive strength at full command and a response rate that limits how quickly the applied command may change — the latter imitating real motors, which cannot jump from stationary to full speed instantly. Tyre-to-floor friction was calibrated by trial and error until the car behaved plausibly, and an early tunnelling defect, in which fast-moving bodies passed through obstacles, was resolved through the rigid-body and collider configuration.

A decision-requester component asks the policy for an action once every five physics steps rather than every step, giving an effective control rate of about 10 Hz at Unity's default physics rate. This figure matters beyond the simulation, because a policy asked for decisions far more or less often than it was trained with behaves differently even when the network is identical; the vehicle's inference loop was set to run at a comparable rate for this reason.

7.3 Observation Space

The agent observes an eighteen-value vector: the unit direction to the target (three values) and the target distance (one); the car's own local linear velocity (three) and yaw rate (one); the two ultrasonic readings (two); the six infrared readings (six); and the current smoothed state of the two motors (two). Everything in the vector is normalised or clamped, so no single input can dominate the network's input scale.

The simulated sensors are modelled as raycasts rather than as physical devices. Each ultrasonic range-finder is a fan of fifteen rays spread over a thirty-degree cone, reporting the shortest distance to any obstacle the fan strikes, normalised to the sensor's five-metre range. Each infrared sensor is a much narrower six-ray bundle over six degrees, and, like the real modules, reports only a binary result: something is close, or it is not. Modelling the cone rather than a single ray matters, because a single ray slips past the corner of an obstacle that a real ultrasonic pulse would detect.

7.4 Action Space

The most important design decision in the training setup concerns the action space. The obvious choice for a differential-drive robot is to let the network command the left and right motors independently, but this trains badly: to turn on the spot, the network must discover by chance that two independent outputs should take opposite signs at the same instant, which is rare early in training when its outputs are close to random. The action space was instead defined as throttle and steer, mixed into left and right commands in code, so that an in-place turn is a single push on one axis. Turning was discovered almost

immediately afterwards. This is also exactly what a real differential-drive controller does, so the change made the simulation closer to the target hardware rather than further from it, and the same mixer is reproduced on the vehicle.

Listing 7.1: Mixing throttle and steer into left/right commands, with a rate limit modelling real motor lag.

```
float throttle = actions.ContinuousActions[0];
float steer = actions.ContinuousActions[1];

// A single push on the steer axis produces an in-place turn, so exploration
// finds turning immediately instead of waiting for two independent outputs
// to happen to disagree.
float targetLeft = Mathf.Clamp(throttle + steer, -1f, 1f);
float targetRight = Mathf.Clamp(throttle - steer, -1f, 1f);

// Real motors cannot change speed instantly; approach the commanded value
// at a bounded rate so the policy learns against realistic dynamics.
smoothedLeft = Mathf.MoveTowards(smoothedLeft, targetLeft, motorResponseRate *
    Time.fixedDeltaTime);
smoothedRight = Mathf.MoveTowards(smoothedRight, targetRight,
   motorResponseRate * Time.fixedDeltaTime);
```

7.5 Reward Engineering

The reward is shaped rather than sparse. A purely terminal reward — one point for reaching the target, minus one for a collision — gives the agent almost no information during an episode, and from a random start it may never reach the target at all. Instead the agent is rewarded continuously for reducing its distance to the target and for reducing its heading error, both measured as an improvement over the previous step rather than as an absolute value. Penalties discourage the behaviours a distance reward alone would permit: approaching obstacles, tripping an infrared sensor, reversing unnecessarily, and changing the commanded action violently from one step to the next. A small penalty is applied on every step, which makes hesitation costly and rules out the standing-still solution that a naive reward invites.

That standing-still solution was a real early failure and shaped the team's approach to reward design: an agent penalised for hitting walls learned simply to stop moving, which satisfied the penalty perfectly while ignoring the goal. The lesson taken from it — that a reinforcement learning agent optimises exactly what it is told to optimise, so an unexpected behaviour is usually a correct solution to a badly written specification — led the team to treat the reward as a specification to be reviewed for loopholes rather than a

set of knobs to be tuned.

7.6 Curriculum Learning

Even with a shaped reward, training the full task at once did not work well, so training was staged as a curriculum. The first curriculum addresses heading: the car initially spawns roughly facing its target, so turning is only a small correction, and as its average reward passes a threshold the permitted spawn heading widens, until the target may be anywhere, including directly behind. Once heading was solved in an empty arena, obstacle avoidance was trained as a second curriculum, warm-starting from the empty-arena network rather than beginning again, and raising the number of active obstacles in stages from none to a fully cluttered room.

One detail of the obstacle curriculum is counter-intuitive and worth recording. The natural assumption is that each stage should demand a higher reward than the last, as the heading curriculum did. For obstacles the opposite holds: adding obstacles makes the task harder, so routes lengthen and collisions become more frequent, and the achievable mean reward *falls* as the curriculum progresses. A rising reward threshold would therefore stall the curriculum permanently at an early stage — an error that looks like a training failure rather than a configuration mistake. The obstacle curriculum was instead advanced on training *progress*, so each density holds for a fixed share of the run regardless of what the reward does. The deployed configuration ramps obstacle count across six stages — clear, then six, fifteen, twenty-five, thirty-five and a full forty-three obstacles — each gated on a fraction of total training progress.

7.7 Domain Randomisation

Several features of the environment exist specifically to stop the policy depending on precision the real hardware cannot deliver. Gaussian noise is added to every ultrasonic reading, and each infrared sensor has a small chance per reading of reporting the opposite of the truth, mimicking the false positives and dropouts these inexpensive modules produce in practice. The car's own mass and the force its motors can apply are each re-randomised by up to fifteen per cent at the start of every episode, so the policy cannot become tuned to one exact vehicle. Obstacles are placed procedurally each episode, subject to clearance buffers around the car and the target so that no episode begins in an unwinnable position, and the target itself moves to a random location. The motor rate limit described earlier means the policy is trained against motors that respond gradually, as real ones do.

Domain randomisation has a limit worth stating, because it bears directly on the vehicle's main open problem. Randomisation fixes *parametric* mismatch, where the

simulator has the right structure but uncertain values, and it can express an uncertain parameter as a range rather than resolve it by measurement. It cannot fix *structural* mismatch, where the simulation supplies a quantity the real system cannot measure at all. The vehicle's missing velocity measurement, discussed in Chapter 8, is exactly this second case.

7.8 PPO Training

The policy was trained with Proximal Policy Optimisation [Schulman et al., 2017], the default on-policy algorithm supplied with ML-Agents. PPO is an actor-critic policy-gradient method that limits how far the policy can move in a single update, trading the fastest possible step for steady improvement — which matters more than sample efficiency on a project where training runs are long and failures costly to repeat. The network is a small two-layer fully-connected network of 128 units per layer, with observations normalised, trained with a learning rate of 3×10^{-4} on linear schedules, a discount of 0.99 and the other hyper-parameters set as in the training configuration reproduced in Appendix C.

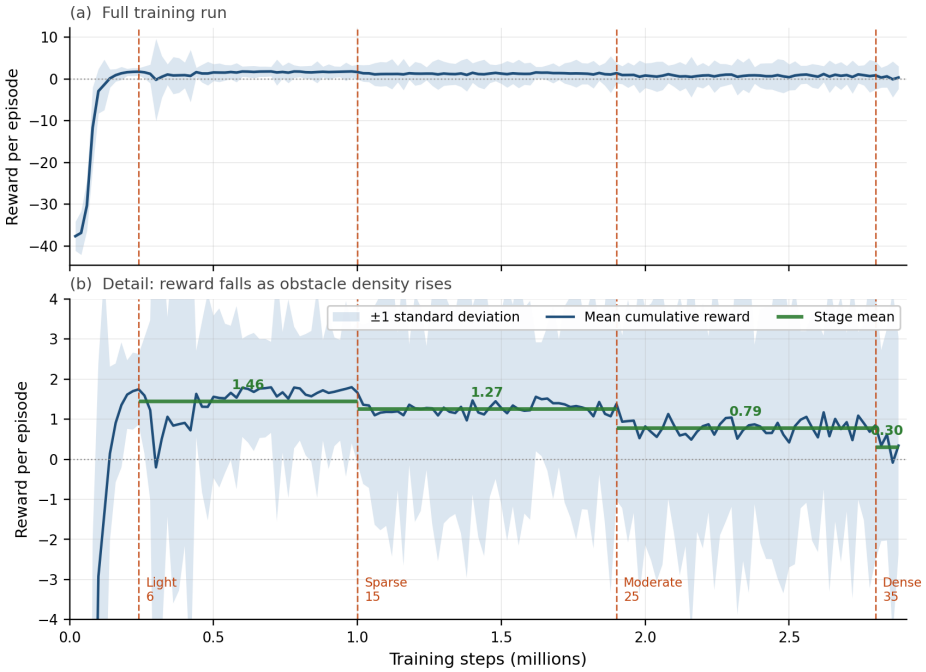


Figure 7.1: PPO training: reward per episode across the obstacle curriculum.

The final run that produced the deployed policy ran for approximately 2.88 million steps. Learning was fastest at the very beginning: the agent started at a mean reward of about -37.7, dominated by collisions and timeouts, and reached positive reward within roughly 150,000 steps. Thereafter the curve is best read stage by stage, and the mean reward within each obstacle stage falls steadily as density rises — around 1.46 with six

obstacles, 1.27 with fifteen, 0.79 with twenty-five, and 0.30 in the densest stage. This is the concrete form of the point made above: a policy that is not getting worse can still earn less, because the task it is being asked to perform is harder.

Because reward scale is arbitrary, performance is reported by episode outcome instead. One episode-handling detail supports the training data's honesty: an episode that times out is *interrupted* rather than ended, so the value of the final state is estimated rather than assumed to be zero. Ending it normally would tell the algorithm that a state in which the car was simply still driving had no future value, which is untrue, and would let a merely slow or wedged car poison the training data. The measured outcomes at the deployed checkpoint are reported in Chapter 11.

7.9 Policy Export and Deployment

The trained network is exported in the ONNX format, an open interchange format that separates the training environment from the deployment environment, so the Unity toolchain need not exist on the vehicle. On the Jetson the network is loaded by an inference runtime and executed on the CPU, which has ample headroom for a small two-layer network at 20 Hz; hardware-specific compilation through TensorRT is a drop-in option for later but is not needed for a network this size, and would consume GPU capacity the vision pipeline will want. How the exported policy is run on the vehicle, and matched to its training conditions, is the subject of Chapter 8.

7.10 Sim-to-Real Considerations

A policy trained in simulation is only useful if it survives contact with a real robot, and the usual reason it does not is that the simulation is too clean. The randomisation and sensor noise above address the parametric side of that gap. A second transfer question — whether the obstacle avoidance learned in Unity works on the physical car at all — was addressed by a dedicated variant of the agent that drives forward and dodges with no destination. Its observation vector is cut to only what the rover can actually measure — yaw rate, the two ultrasonic readings, the six infrared readings and the controller's own motor state, eleven values in place of eighteen — dropping the target direction and distance, which need a map or GPS the car does not have, and the local velocity, which needs odometry it also does not have. Because its observations are exactly the car's real senses, there is no sim-to-real observation gap left to discover on the floor. Its reward is placed on *measured* forward speed rather than commanded throttle, since paying for commanded throttle would reward a car pinned against a wall at full throttle as richly as one crossing the room; and a timed-out episode, meaning the car survived, is the

good outcome. This transfer test had not yet been trained or driven at the close of the placement, and is listed as future work in Chapter 13.

8 Embedded Software and Safety Architecture

This chapter describes the software that runs on the vehicle: the control and telemetry stack, the motor control at its core, the runner that executes the trained policy, and — most importantly — the safety layer through which every command must pass. Because two motors share each L298N channel of a driver with no thermal protection of its own, this software is the thermal protection, and it is treated accordingly.

8.1 Embedded Software Stack

The onboard software is a single Python application built on Flask with WebSocket support, running as a system service. It is organised into modules with separated responsibilities: a server owning the interface and control arbitration; a motor module running the safety loop; a sensor module sampling and converting; a policy module running inference; a hardware module providing GPIO, PWM and I^2C access; and a configuration module holding every pin, constant and limit, so that no value is duplicated elsewhere. All hardware access passes through the single hardware module, which is what made the change of computing board tractable: moving from the Raspberry Pi to the Jetson meant writing a new backend against that one narrow interface — set a pin, read a pin, set a PWM duty, talk to the I^2C bus — and leaving everything above it, including the entire safety layer, unchanged. The same abstraction provides the dry-run mode in which the whole server runs on a laptop with no hardware attached, where most logic faults were found before they could reach the vehicle.

8.2 Control and Telemetry Architecture

A single WebSocket carries commands down and telemetry up; HTTP serves only the static control page. Any number of clients may connect and watch telemetry, but only one may hold control at a time, arbitrated through a small `take/release/command/reset` protocol, so two operators cannot fight over the vehicle. Commands are sent as JSON at 20 Hz carrying per-side values in the range $[-1, 1]$ and a sequence number; values are clamped on arrival and malformed packets are dropped rather than allowed to crash the

loop. The client sends continuously, *including zeros when idle*, because silence is what fires the deadman.

Two networking details are worth recording because each was learned from a fault. Any packet older than the last one accepted is discarded, since WebSocket delivery can reorder under load and applying a stale command means driving on out-of-date intent. And when control changes hands the sequence counter is reset — a detail that surfaced only through a bug in which a newly connected controller found every one of its packets rejected as stale, leaving the car immobile with the deadman latched. It is a clean example of two individually reasonable decisions combining into a fault that neither predicts.

Telemetry is published as one merged frame at 20 Hz rather than one message per sensor, carrying the raw inertial counts, the two ultrasonic distances, the infrared bitmask, the duty actually applied to each channel, and the deadman and stall flags. The browser client is framework-free: it presents a touch joystick for phones and reads a physical gamepad where one is connected, mixes those inputs into left and right wheel commands with a standard arcade mix, and displays the live telemetry together with the state of every guard. It greys out stale telemetry and reconnects with backoff. A USB camera feed was later added, streamed as MJPEG and decoded natively by the browser; it is documented, deliberately, as situational awareness rather than a driving instrument, since the video runs a few hundred milliseconds behind reality — on a poor link, more than the deadman interval — so driving out of line of sight on the camera alone is unsafe with the current stack.

8.3 Motor Control

The motor module is the only code that writes to the motor pins, and it runs the safety loop at 50 Hz. It maps the left channel to one pair of direction pins and an enable pin, and the right channel to the other, with a 2 kHz PWM carrier. One correction in this layer is instructive. On first driving, the controls behaved as though rotated ninety degrees, which resolved into two independent harness faults: the two channels crossed, and one motor pair's leads reversed. The fix could have been applied in the browser's input mixer, and for a human driver that would have worked — but the trained policy commands the motors directly and never passes through the browser, so it would have been left driving a rotated car. The correction was therefore made in the pin definitions, beneath every consumer, so that the browser client and the policy both inherit it. Choosing the layer at which to apply a fix, so that it holds for all callers rather than the one in front of you, is a habit the project reinforced repeatedly.

The motor module also supports a dummy-output mode: when output is disabled, the full safety pipeline still runs and the applied-duty telemetry updates normally, but the H-bridge pin writes are suppressed. This is what allows the policy to be engaged on the

real Jetson, with real hardware attached, and the whole loop observed without the wheels turning.

8.4 ONNX Policy Runner

The policy runner is another controller on the same interface the browser uses. A dedicated thread runs at 20 Hz: it assembles the eighteen-value observation vector, runs one ONNX forward pass, converts the two network outputs into left and right wheel commands, and submits them through the same entry point a human driver's commands pass through. The observation is assembled in the same order and with the same normalisation constants the Unity agent used, the throttle-and-steer mixer is a line-for-line translation of the training code, and the motor-smoothing that Unity applies is reimplemented in Python, because two of the observations the policy reads are its own recent smoothed commands fed back to it. A mismatch in any of these would not raise an error; the network would simply receive inputs subtly unlike those it trained on and behave worse for reasons hard to diagnose.

Listing 8.1: The on-device inference tick: the same mixer as in training, submitted through the same interface a human driver uses.

```
obs, debug = self.obs_builder.build()
out = self.session.run([self._output_name], {self._input_name: obs})[0]

throttle = float(np.clip(out[0, 0], -1.0, 1.0))
steer = float(np.clip(out[0, 1], -1.0, 1.0))

# Mixer - line-for-line from the Unity agent's OnActionReceived.
target_left = max(-1.0, min(1.0, throttle + steer))
target_right = max(-1.0, min(1.0, throttle - steer))

# Advance the smoothed-action memory exactly as Unity does, so the policy's
# own feedback observations on the next tick match what it saw in training.
self.obs_builder.update_smoothed(target_left, target_right, dt)

# Same entry point the browser uses - every safety guard still applies.
self.motor_thread.set_command(target_left, target_right, self._seq)
```

The sources feeding each observation are set out in Table 8.1. One entry is the vehicle's principal limitation: the local-velocity observation is fed zeros, because the car has no wheel encoders or odometry. Feeding zeros is a deliberate and documented choice rather than an oversight — it is the honest value in the absence of a measurement — but the policy was trained with real velocity information and will not behave as it did in simulation

until that information is restored.

Table 8.1: Sources of the eighteen observation values on the vehicle.

Slot(s)	Observation	Source on the vehicle
0–2	Target unit vector	UI-editable local target (distance and bearing)
3	Target distance	Same
4–6	Local linear velocity	**Zero** — no odometry; the main open gap
7	Yaw rate	MPU6050 gyro, converted and normalised
8–9	Ultrasonic front/rear	Range reading normalised to $[0, 1]$
10–15	Infrared $\times 6$	Six-bit mask, order FL, FR, RL, RR, L, R
16–17	Smoothed left/right	Controller-side memory, maintained in Python

Inference is comfortably fast: in dry-run measurement it took a fraction of a millisecond per tick, far inside the loop's budget, so there is no timing pressure on the control rate.

8.5 Safety Layer

The safety layer is the reason the whole architecture is shaped the way it is. Five guards run on every tick of the motor loop, in a fixed order, server-side, and every command — browser, policy or test script — passes through all of them:

1. **Deadman.** If no command has arrived for 300 milliseconds, the target is forced to zero. A dropped connection, a closed laptop lid or a frozen browser stops the car rather than leaving it driving on the last command it received.
2. **Stall latch.** If duty is being commanded but the inertial sensor reports no motion for a second, the channel latches off until a client explicitly resets it. A wheel jammed against a wall is the exact condition that overheats the driver.
3. **Slew limit.** The applied duty ramps toward its target rather than jumping, taking about 200 milliseconds to traverse full scale, which protects the gearbox and the battery from instantaneous current demand.
4. **Zero-cross coast.** A command reversing direction is not applied directly; the channel ramps to zero, coasts for 80 milliseconds, and only then drives the other way, so the H-bridge is never asked to reverse a spinning motor.
5. **Duty cap.** Applied last, clamping the output to 55 per cent. Because it is last, no earlier stage can produce an output above it.

The ordering is itself a design decision: the cap is applied at the end so that it is unconditional, and the deadman at the beginning so that a lost link overrides everything downstream of it.

Listing 8.2: The safety pipeline. Every command, browser or policy, passes through these stages in this order, server-side.

```
# 1. deadman - no packet for DEADMAN_S -> target zero
self.dead = (now - last_cmd) > config.DEADMAN_S
if self.dead:
    tgt = {"L": 0.0, "R": 0.0}

# 2. stall latch - duty commanded but no IMU motion -> latch off until reset
if config.STALL_GUARD_ENABLED and self.stall:
    tgt = {"L": 0.0, "R": 0.0}

for ch in ("L", "R"):
    want = tgt[ch]
    # 4. zero-cross - ramp to zero and coast before reversing
    if cur[ch] * want < 0 or now < self._coast_until[ch]:
        want = 0.0
    # 3. slew limit - bounded change per tick
    step = max(-max_step, min(max_step, want - cur[ch]))
    self._applied[ch] = cur[ch] + step
    # 5. duty cap - applied LAST, so nothing upstream can exceed it
    out = max(-config.DUTY_CAP, min(config.DUTY_CAP, self._applied[ch]))
    self._drive_channel(ch, out)
```

Two honest qualifications apply to this layer. The stall-latch threshold has never been validated against real inertial data — it was chosen while no IMU was fitted — and the guard senses *chassis* rotation rather than *wheel* rotation, so it can miss a stall in which the wheels slip while the body is still, and can false-trip with the car on blocks. For anything motor-related the guard actually relied upon is therefore the duty cap. A current-sense stall detector, prototyped in one of the Python simulators, remains the better primitive and is listed as future work.

8.6 Risk Assessment

The project's risks cluster around the same physical facts. The dominant risk is thermal: a sustained stall on the paralleled-motor driver, mitigated by the duty cap primarily and the stall latch secondarily. Power-rail collapse under a stall transient is mitigated by the dual-battery isolation — which, as noted in Chapter 5, is designed and built but not yet operating, so this risk is currently carried rather than closed. The risk of a learned policy commanding an unsafe manoeuvre is mitigated structurally: the policy is a driver, not a privileged one, so a manoeuvre it has learned cannot be executed any more than a human

could push it past the same guards. A subtler risk found during the Jetson bring-up was a PWM initialisation ordering quirk that could leave an enable pin stuck fully on, bypassing the duty cap; it was closed by pairing each pin's setup with the construction of its PWM object so no caller can get the order wrong, and by booting the pins to the off state.

The Autonomy Lab adds a set of interlocks around live testing: engaging the policy is refused while a human holds control, and refused on real motors unless the client has confirmed the wheels are off the ground — a gate the server enforces rather than trusting the interface; human control is refused while the policy is engaged, since the policy is the driver; a WebSocket disconnect tears down the runner and the deadman stops the car within its interval; and any exception during inference disengages the policy and lets the deadman catch whatever slips through.

8.7 Deployment on Jetson Orin Nano

The application is installed as a system service so the vehicle is drivable from power-on without anyone logging in. Bringing it up on the Jetson required enabling hardware PWM on the correct header pins through the board's pin-mux tool — pins that had to be confirmed on the hardware, since several published pinouts were wrong for this board — and confirming the inertial sensor on the correct I^2C bus, which on the Orin Nano is bus 7 rather than the bus 1 that most Raspberry Pi and older-Jetson guides assume. The service starts in the hardware sensor mode with real motor output, and the bring-up checklist is worked with the wheels off the ground: the deadman is verified to stop the wheels within its interval and the direction mapping is confirmed before the vehicle ever touches the floor. One property of the current design is recorded rather than changed: because the deadman is a loop inside the process, if the process itself dies the pins hold their last duty until the service manager restarts it, a short interval of uncommanded driving that a hardware failsafe would remove and that is noted as future work.

9 Computer Vision

This chapter describes the computer-vision strand of the project. It is deliberately modest in its claims: vision on EAMS is an early prototype, pursued as groundwork for future semantic perception rather than as a subsystem in the control loop, and this chapter says so plainly.

9.1 Role of Vision in EAMS

The vehicle's navigation is driven by the ultrasonic and infrared sensors and the trained policy; the camera plays no part in it. Vision was pursued as a separate, forward-looking strand — internally the project's third work package — whose purpose is to lay the groundwork for semantic scene understanding, the kind of perception the longer-term goal of natural-language navigation will eventually need. Where the ultrasonic and infrared sensors answer "is something close?", a camera can begin to answer "what is it?", and that distinction is the reason the strand exists. At its current stage, however, it produces situational awareness for the operator and a proof of concept, not an input to any control decision.

9.2 Camera and Edge Deployment

A USB webcam is mounted facing forward. It was chosen for availability and simplicity, and it serves two roles: giving the operator a view from the vehicle, and providing frames for the vision prototype. The camera subsystem was written to the same contract as the rest of the optional software — it must be incapable of stopping the pilot stack from booting. A missing software dependency, an absent device or a cable pulled mid-drive all log the fault and retry with backoff rather than bringing anything down. The video is captured by a dedicated module and served as an MJPEG stream that the browser decodes natively, so no client-side decoding library is required, with separate endpoints for a single snapshot and for status. Camera status is a separate poll rather than a field in the telemetry frame, deliberately, so that camera trouble can never perturb the control frame. One capture thread serves any number of viewers and the device is released after a short idle period to save power and USB bandwidth, and the feed defaults to off, because it shares the wireless link with the command stream and on a weak link the sensible choice is responsive control over video.

9.3 Vision Pipeline

The prototype implements person detection: frames captured from the camera are passed through a detector running on the Jetson, and detections are drawn on the output frame. The pipeline is kept simple and runs on the vehicle's own computer, consistent with the project's edge-inference stance that a mobile platform cannot depend on a network round-trip of unbounded delay. Where the navigation policy is a small network that runs comfortably on the CPU, a vision model is the kind of convolutional network that benefits substantially from hardware acceleration, which is noted in the roadmap below.



Figure 9.1: The vision prototype detecting a person through the vehicle's camera.

9.4 Prototype Results

At its current stage the prototype detects people in the camera's view and marks them on the streamed frame. This demonstrates the capture-to-inference path end to end on the edge device and establishes that a vision model can be run on the vehicle alongside the control stack. It is a proof of concept rather than a measured capability: it has not been characterised for accuracy, frame rate under load, or robustness across lighting, and no such figures are claimed here.

9.5 Relationship to Autonomous Navigation

Two properties keep vision out of the driving loop for now. First, it is not wired into the policy: the network's eighteen observations come from the ultrasonic, infrared and inertial sensors, and adding a visual input would mean retraining the policy against a simulated camera, which the project has not done. Second, and more fundamentally, the streamed video runs a few hundred milliseconds behind reality — on a poor link, more

than the deadman interval itself — so it is explicitly situational awareness and not a driving instrument. Driving the vehicle out of line of sight on the camera alone would not be safe with the current stack, and the code records that limitation beside the feature so it cannot be forgotten.

9.6 Future Vision Roadmap

The strand points in a clear direction. The immediate technical step is to accelerate the vision model through the Jetson's dedicated inference tooling, which is where the platform's GPU earns its place and is the reason a Jetson was specified rather than a smaller computer. Beyond that, the camera can be developed from an operator aid into an actual sensor: lightweight overlays on the frame, such as the target bearing and the infrared state, are a small step, since the capture path already decodes each frame. The larger ambition is semantic scene understanding that could feed the navigation stack — recognising what an obstacle is, not merely that it is there — and, further out, the vision-language perception that the natural-language goal depends on. Each of these is future work; the contribution of this placement was to establish the edge capture-to-inference path and confirm that a model runs on the vehicle at all.

10 Implementation, Integration and Testing

This chapter follows the project from bare hardware to a running vehicle through the phase gates the team worked to. It records the bring-up of each computing board, the integration testing carried out before anything drove real motors, the faults met along the way and how they were resolved, and the demonstrations given during the placement.

10.1 Development Phases

Work moved through a sequence of phase gates, each of which had to pass before the next began. The order was chosen so that the riskiest step — driving real motors on a marginal driver — came last, after everything upstream of it had been proven.

Table 10.1: Development phase gates.

Phase	What it established
Laptop dry-run	The whole server, client and policy loop running with GPIO stubbed
Raspberry Pi bring-up	The control stack and safety layer proven on real hardware
Jetson migration	The stack ported to the deployment board, policy running on-device
Wheels-off-ground	PWM, direction mapping, deadman and stall guard verified with wheels free
Floor driving	The vehicle driven under power once all guards were confirmed

10.2 Hardware Assembly and Bring-up

The physical platform was rebuilt several times over the placement: rewired on arrival, moved from breadboard to soldered stripboard for reliability, and given a new computing board when the project moved from the Raspberry Pi to the Jetson. Electrical bring-up was staged one subsystem at a time — the power rail, then a single infrared sensor, then all six, then the ultrasonic pair under sequential triggering, then the inertial sensor, then motor PWM, and finally full integration — so that a fault in any one subsystem could be found in isolation rather than in a fully assembled vehicle. Several electrical preconditions were established before power was applied to the motors: the L298N's onboard 5 V

regulator jumper was removed, the sensor supply was moved to the board's 3.3 V rail so no signal line exceeded the header's rating, and a star ground was used so motor return current is kept out of the logic ground.

10.3 Raspberry Pi Pre-Validation

The vehicle was first built around a Raspberry Pi Zero 2 W, and this pre-validation phase proved the entire control stack before the Jetson was available. It was not without friction. On the Pi's current operating system the pin-control library the software depends on is no longer distributed as a package, so it was compiled from source and its background daemon registered as a system service manually; the Python dependencies were installed into a virtual environment; and the I^2C interface was enabled for the inertial sensor. With the wheels off the ground, the server came up, the browser client loaded, the WebSocket streamed telemetry, and the deadman and stall-and-reset behaviour were all verified on real hardware; the car was then driven under power, confirming the direction mapping and control on the actual vehicle. Because the Pi's wireless connection proved unreliable enough to make deployment frustrating, the board was configured to present itself as a USB Ethernet device, giving a stable link over a single cable independent of any wireless network — infrastructure work that produced no visible feature but turned a slow deploy-and-test cycle into a fast one. Through this phase the inertial sensor was not yet fitted, so the vehicle was driven with the stall guard disabled, a documented and accepted operating configuration at the time rather than a silent failure.

10.4 Migration to Jetson Orin Nano

The Jetson was introduced for one reason: it has a GPU, and the Pi Zero 2 W does not have the computing power to run a neural-network policy while also driving the car.

Flashing the board was the single most time-consuming problem of the placement and consumed the better part of a week. The vendor's flashing tool supports only Ubuntu and Windows hosts, and the development machine runs Fedora, so the first attempt used an Ubuntu virtual machine, which could not hold the USB connection to the board through the virtualisation layer. The path that eventually worked required removing that layer entirely: installing Ubuntu on real storage, cloning it onto an internal partition when external media proved too slow and too small, creating the boot partition by hand, and only then flashing the board over a direct USB connection, which brought the flash down to about ten minutes. The chain of failures along the way — a tool that would not launch under the host's display server, storage that was undersized or undetected, installers that offered to overwrite a partition that had to be preserved — is recorded in more detail in

the appendices, but the general lesson is that vendor tooling is built and tested against a narrow set of environments, and everything outside that set becomes the user's problem.

Porting the code was, by contrast, comparatively small, because all hardware access was already funnelled through one narrow interface. A new backend was written against that interface using the Jetson's libraries and pin-numbering convention, and everything above it, including the entire safety layer, moved across unchanged — the clearest practical demonstration in the project of why isolating platform-specific code behind an interface is worth the effort.

The motors that would not run on the Jetson took several days and turned out to be two independent faults at once, which is why isolating either one alone never produced a working system. The board's pins had been left in a mode other than general-purpose output, corrected with the board's pin-mux tool; and the two PWM enable pins were being initialised in an order that left only one of them actually driving, which was fixed by pairing each pin's setup with the construction of its PWM object so that a caller cannot get the order wrong. The second fault mattered beyond mere function: a stuck-fully-enabled channel bypasses the duty cap, which is the protection standing between the software and an overheating board, so a library ordering quirk had a direct safety consequence.

The crossed motor channels surfaced on the first drive as controls that behaved as though rotated ninety degrees, and resolved into two harness faults — crossed channels and one reversed motor pair — corrected in the pin definitions beneath every consumer, so that both the browser client and the policy inherit the fix.

Service, camera and inertial sensor. The application was installed as a system service, after an initial path error that presented as a restart loop rather than a Python fault. A USB camera feed was added for operator video. Finally the inertial sensor was brought up, which had been failing not because it was absent but because it was being addressed on the wrong I^2C bus number — bus 1, carried over from the Raspberry Pi, where the Orin Nano's header pins are in fact bus 7. With the sensor reading, the stall guard was re-enabled, closing a risk the project had carried since the Pi phase.

10.5 Integration Testing

Most integration testing was done off the vehicle or with the wheels off the ground, in keeping with the phase-gate order. The dry-run mode ran the whole server, client and policy on a laptop with GPIO stubbed. Two Python simulators — a broad digital twin and a focused rover simulator that mirrors the real vehicle's telemetry and WebSocket architecture — let a controller drive a simulated rover through the same interface as the real one, and the focused simulator was used to exercise a current-based stall detector that the real vehicle does not yet carry. On the vehicle itself, the Autonomy Lab substitution modes allowed the full autonomous pipeline to be run with the policy engaged on the

real Jetson but the motor output suppressed, so the loop could be observed without the wheels turning. In a representative dry-run of the autonomous pipeline the policy loaded and ran with inference taking a fraction of a millisecond, the duty cap was seen to bite correctly on the more heavily commanded channel, the kinematic pose advanced as expected, the deadman fired within its interval when the connection closed, and a sweep of target directions confirmed the sign conventions end to end.

10.6 Faults and Resolutions

Table 10.2 collects the faults that cost real time or changed a decision. A recurring pattern across them is that the visible symptom frequently pointed at the wrong subsystem — a sensor reading nothing because a pin was multiplexed away from general-purpose use, motors not running because of the order two objects were constructed in, training runs dying because a window had lost focus — so the method that resolved most of them was to reduce the number of things that could be responsible until only one remained.

Table 10.2: Selected faults encountered and their resolution.

Fault	Resolution
Pin-control library not packaged on the Pi's OS	Compiled from source and its daemon registered as a service by hand
ML dependency resolution failed; Python too new	Switched resolver, pinned a working Python and package set, recorded for reproducibility
Training runs timed out on window switch	A "run in background" setting was disabled; enabling it fixed it completely
Duplicated simulated sensors and an extra observation	Found by comparing editor configuration against the agent script; removed
Agent would not learn to turn	Changed the action space from independent wheels to throttle and steer
Jetson flashing (week-long chain)	Removed the virtualisation layer; installed to internal storage; flashed over direct USB
Pi wireless link unreliable	Configured the Pi as a USB Ethernet gadget for a stable single-cable link
Motors would not run on the Jetson	Two faults: pins left in the wrong mode, and PWM init ordering; both corrected in the hardware backend
Front ultrasonic silently read nothing	A trigger pin was muxed to the PWM controller; left as plain GPIO in configuration

Table 10.2 – continued

Fault	Resolution
Controls rotated ninety degrees	Crossed channels and a reversed pair, corrected in the pin definitions beneath every consumer
New controller immobile, deadman latched	Sequence counter reset on control handover so fresh packets are not rejected as stale
Sensors driven at 5 V into 3.3 V pins	Moved the sensor supply to the board's 3.3 V rail; no level shifting needed
Inertial sensor appeared absent	It was on I^2C bus 7, not bus 1; corrected, and the stall guard re-enabled

10.7 Demonstrations

The project was presented for critique at the end of several weeks, both to the other interns and to supervisors, which meant explaining technical work to people who were not working on it. It was also demonstrated during a visit to the laboratory by the Minister of ICT, where the vehicle was shown and its operation explained. Preparing to present repeatedly proved useful beyond the presentations themselves, since it reliably exposed the parts of the work that were less well understood than assumed.

11 Results and Discussion

This chapter reports what was achieved against the objectives set in Chapter 3, subsystem by subsystem, and discusses the results honestly — distinguishing throughout between what was designed, what was built, what was simulated, and what was measured on the vehicle.

11.1 Achievement of Objectives

Table 11.1 summarises the status of each objective. The pattern across it is that the software and simulation objectives were largely met, while the objectives that depend on the full physical sensor and power suite were partially met, with the remaining work clearly identified rather than obscured.

Table 11.1: Achievement of project objectives.

Objective	Status
Differential-drive testbed with a low-level safety layer independent of the AI stack	Achieved; vehicle built and the safety layer proven on hardware
Train a navigation policy with PPO across an obstacle curriculum and export it	Achieved; policy trained and exported to ONNX
Transfer the trained policy to the physical vehicle	Partially achieved; policy runs on-device and has driven the real motors, on substituted sensor data
Sensor fusion for pose estimation, validated against dead reckoning	Achieved in simulation; validated in MATLAB, not yet deployed on the vehicle
Power and drive electronics against worst-case, with isolation and protection	Designed and built; the dual-battery isolation is not yet operating
Prototype an edge vision capability	Achieved as a prototype; person detection, not in the control loop

11.2 Electronics Results

The current analysis confirmed the central electrical concern: with two TT motors paralleled per channel, the worst-case stall current approaches the practical limit of a single L298N channel, and the driver has no current limiting or thermal shutdown of its

own. The 55 per cent duty cap is the design response, holding the commanded output within the driver's thermal envelope under sustained load. The Proteus simulations of the battery-management and motor-drive blocks agreed on rail voltages before either went to hardware, and the sensor supply was moved to the 3.3 V rail so that no signal line exceeds the header's rating. Two electronics results are reported as incomplete: the real per-motor stall current has not been measured, so the duty cap remains an estimate that may need to come down; and the dual-battery isolation, though designed and built, is not operating, since the compute pack is unconnected and the Jetson ran from a mains adapter.

11.3 Sensing and State Estimation Results

The pose Extended Kalman Filter was validated in MATLAB against a simulation constructed to create, deliberately, the same prediction error the real vehicle suffers. Under simulated wheel slip, the filter held position error bounded within roughly 0.01 to 0.07 m, peaking near 0.07 m and returning to about 0.02 m, and — the result that matters — fluctuating without trending upward, where unaided dead reckoning under the same conditions would accumulate error without bound. The combined simulation also surfaced a limitation rather than hiding it: in one configuration the reactive obstacle layer became trapped in a local minimum and did not reach the goal, the characteristic failure of purely reactive navigation, which a learned policy does not escape simply by being learned. The filter is a validated design; it is not yet integrated into the vehicle, and estimated pose currently feeds nothing on the car.

11.4 Reinforcement Learning Results

At the deployed checkpoint the policy reached its target in about 79.7 per cent of episodes in the densest obstacle configuration, collided in about 20.3 per cent, and timed out in effectively none. The success rate is quoted in preference to reward because reward scale is arbitrary, whereas a success rate means the same thing to any reader; it is close to certain in an empty arena and falls to roughly four episodes in five once the room is fully cluttered. The near absence of timeouts is itself a result: it indicates the car commits to a route rather than hesitating, which is the behaviour the per-step penalty was introduced to produce. Read across the obstacle curriculum, the per-stage mean reward falls as density rises, the expected consequence of a harder task rather than a failing policy, and the reason the curriculum was advanced on progress rather than on reward.

11.5 Embedded Software and Safety Results

The safety layer was verified to behave correctly on hardware: the deadman stops the wheels within its interval on a lost connection, the stall latch cuts and holds until an explicit reset, and the duty cap clamps the applied output. Crucially, the same guards were confirmed to apply identically to browser and policy commands, since both pass through one interface; the policy is a driver, not a privileged one. The trained policy loads and runs on the Jetson with inference taking a fraction of a millisecond, well inside the loop budget, and the Raspberry Pi-to-Jetson port carried the entire safety layer across unchanged. Two results are qualified honestly: the stall-guard threshold has not been validated against real inertial data and the guard senses chassis rather than wheel rotation, so for anything motor-related the duty cap is the guard actually relied upon.

11.6 Computer Vision Results

The vision prototype detects people in the camera's view and marks them on the streamed frame, demonstrating the capture-to-inference path end to end on the edge device. It is a proof of concept and has not been characterised for accuracy, frame rate under load, or robustness, and no such figures are claimed. It is not connected to the navigation policy.

11.7 Overall System Performance

Taken together, the results describe a working, well-instrumented autonomy testbed. A policy trained entirely in simulation has been transferred to the vehicle and drives its real motors through the same safety-checked interface a human uses; the safety architecture holds regardless of who or what is driving; and the supporting subsystems — power, sensing, state estimation and vision — are designed and, in most cases, built. What has not yet been achieved is genuine autonomous navigation on the full live sensor suite. The policy has driven the motors on substituted sensor data, not on the complete set of real sensors, and the single change that would most advance the system is the one identified repeatedly below.

11.8 Limitations

Table 11.2: Principal limitations of the system in its present state.

Limitation	Consequence
No wheel encoders / odometry	The policy's local-velocity observation is fed zeros, the largest sim-to-real gap; the EKF prediction can only use commanded motion
EKF not deployed	Validated in MATLAB only; estimated pose feeds nothing on the vehicle
Dual-battery isolation not operating	Compute pack unconnected, Jetson on mains; isolation untested under stall
Stall-guard threshold unvalidated	Never checked against real inertial data; senses chassis, not wheel, rotation; the duty cap is relied upon instead
Real stall current unmeasured	The 55 per cent duty cap is an estimate that may need to come down
Vision is a prototype	Person detection only, not characterised, not in the control loop
Reactive navigation	Susceptible to local minima, as the combined EKF simulation showed
Eight-week window	Bringing the full sensor suite into the policy loop remained unfinished at the close

12 Group Internship Experience

This chapter steps back from the system to reflect on the placement as an experience: the environment it took place in, the skills the team developed, the challenges it met collectively, and the lessons that will outlast this particular project.

12.1 Working Environment

The placement was research-based and largely self-directed. The team was given hardware, a single goal and periodic guidance, and left to work out the method, so a substantial part of the engineering was deciding what to build next and why. The days were structured — a stand-up each morning to report progress and blockers, project-build time through the day, and a logged wrap-up — and leadership of the week rotated around the team, so coordinating the work was itself a shared responsibility rather than one person's job. The team spanned software, electrical and mechanical backgrounds, and much of the value of the environment came from that mix: the project could not be built by any single discipline, and progress depended on people explaining their own work and understanding everyone else's. Work was presented for critique at intervals, including a demonstration to the Minister of ICT, which repeatedly served to expose whatever was less well understood than assumed.

12.2 Technical Skills Developed

The technical range was wider than a placement in any one discipline would have offered. The team learned reinforcement learning with Unity ML-Agents from nothing — observations, actions, rewards, how a policy is represented and improved, how to shape a reward and stage a curriculum, and how to read training metrics critically. It developed fail-safe concurrent software, in which being wrong damages hardware, and learned to design guards that cannot be bypassed and to order them deliberately. It prepared two single-board computers from bare hardware, including building a library from source, registering system services, and a week of flashing a development kit. It implemented and validated sensor fusion with an Extended Kalman Filter, designed and simulated power electronics, and built an edge vision prototype. And it carried a policy trained in simulation across to real hardware, which is the discipline of sim-to-real transfer met in practice rather than in the abstract.

12.3 Engineering and Professional Skills

Beyond the specific technologies, the placement built habits that generalise. Documentation came to be treated as part of the engineering rather than an afterthought: a progress logbook that records why each decision was taken, deployment and setup guides precise enough to follow, and explanatory comments written into the configuration file beside the values they justify, so that the reasoning is met where it matters. Version control moved the team from passing code around informally to a shared repository in which it was always clear which version was on the vehicle. Presenting technical work to people not involved in it became a regular skill rather than an occasional one, as did the harder skill of working across disciplines — asking a useful question about a power supply, or understanding an answer about a motor driver, when one has been trained in only one of the fields involved.

12.4 Challenges Encountered as a Team

The difficulties, taken together, fall into a few causes. Working toward a goal without a prescribed method was itself a challenge, since deciding what to build next was part of the work and some of those decisions necessarily turned out to be wrong. Many faults presented in the wrong domain — a problem that appeared in software but was caused by hardware, or the reverse — so the visible symptom often pointed at the wrong subsystem. Several fields central to the project, reinforcement learning in particular, were not part of anyone's formal training and had to be learned from informal sources while judging whether an explanation was correct without yet knowing enough to be sure. Vendor tooling that assumed environments the team did not have cost a disproportionate amount of time. And working across disciplines without a shared vocabulary took real effort that had nothing to do with the difficulty of the underlying ideas. Underlying all of these, the eight-week window was short for a project of this scope; running the simulation and the physical build as parallel strands was what made it possible to get as far as the team did.

12.5 Lessons Learned

A handful of lessons recurred often enough to be worth stating as principles. Isolating platform-specific code behind one narrow interface is worth the effort, a lesson learned by being paid back for it when the computing board changed and the safety layer moved across untouched. A specification is best read looking for the cheapest way to satisfy it rather than the way it is intended, since a reinforcement learning agent, like any system, optimises exactly what it is told to. A fault should be fixed at the layer where the

correction holds for every caller, not where its symptom happens to appear. A symptom frequently points at the wrong subsystem, so the reliable method is to reduce the number of possible causes until one remains. And when being wrong damages hardware, the protections belong in software that no client can weaken, ordered so that the unconditional guard comes last and the safety-critical override comes first.

12.6 Relationship Between Learning and EAMS

The placement drew on prior study and prior training in ways that were not always predictable. Some of the most useful moments came from the least expected connections — hardware literacy from earlier, more traditional engineering work is what prompted the team to check the sensor datasheets against how the vehicle had actually been wired, which is how it found that eight signal lines were being driven above the voltage the boards accept. More broadly, the project exercised material from across the team's programmes: embedded systems and operating-system concepts in the concurrent control software, modelling and simulation in the digital twin, control and estimation theory in the pose filter, power electronics in the drive stage, and data-communication principles in the telemetry link. Equally instructive was what the curricula had not covered — reinforcement learning, sim-to-real transfer, the practical work of bringing an unfamiliar board into a usable state, and faults that cross the hardware-software boundary — all of which the team met here for the first time and which are the subject of the recommendations that follow.

13 Conclusion and Recommendations

This chapter draws the project together, sets out recommendations for the host organisation and for whoever continues the work, and identifies the future work that would carry EAMS from its present state toward genuine autonomous navigation.

13.1 Conclusion

Over an eight-week placement the team converted an inherited chassis, a single marginal motor driver and a modest compute budget into a working autonomous-mobility testbed. A navigation policy was trained entirely in simulation by Proximal Policy Optimisation, hardened for transfer through domain randomisation and sensor noise, and exported to run on the vehicle's own computer; at the deployed checkpoint it reached its target in about 79.7 per cent of episodes in the densest arena. Control and telemetry software was written for the physical car, structured so that every command — human or learned — passes through one server-side pipeline of five ordered safety guards, because two motors share each channel of a driver that has no thermal protection of its own. Supporting subsystems were designed and, in most cases, built: a dual-isolated power architecture, an Extended Kalman Filter validated in MATLAB, and an edge computer-vision prototype.

The honest position at the close of the placement is that the policy runs on the vehicle and has driven its real motors under its own control, but on substituted sensor data rather than the full live sensor suite, which was still being brought up. Genuine autonomous navigation on real sensors remains ahead, and much of the work described in this report is the scaffolding that will make it testable when the remaining gaps are closed. That framing is not a disappointment but an accurate account of a research project that got a long way in a short time and knows exactly what it has not yet done.

13.2 Recommendations to Pearl AI Labs

The following are offered to the laboratory and to whoever continues the project.

Immediate measurements. Two numbers currently rest on estimates and should be measured. The real per-motor stall current should be measured directly, since the whole safety layer is designed around a limit that has been estimated rather than confirmed, and the duty cap adjusted if it is exceeded. The stall-guard gyroscope threshold should be validated by logging the inertial readings through stationary, straight, turning and

pushed-against-a-wall conditions and setting the threshold from the gap between the stalled and moving cases.

Hardware. Fitting wheel encoders, or otherwise providing odometry, is the single most valuable next step: it would supply the velocity measurement the policy currently receives as zeros and give the pose filter a corrected prediction. The compute battery should be connected and the dual-battery isolation tested under stall, so the vehicle can run untethered as designed. A current-sense stall detector, already prototyped in simulation, would give a stall guard that senses the actual fault condition rather than chassis rotation.

Software. Once encoders exist, the validated EKF should be integrated into the vehicle's sensor loop and the policy's velocity observation fed from a real measurement, which together are what stand between the current system and navigation on live sensors. The obstacle-avoidance transfer test should then be trained and driven on the floor to confirm the learned avoidance survives contact with the real vehicle.

13.3 Future Work

Beyond the immediate recommendations, the project has a clear longer arc. The velocity-observation gap should be closed and full navigation on the live sensor suite demonstrated. The vision strand should be accelerated on the Jetson's inference hardware and developed from an operator aid toward semantic scene understanding. A hardware failsafe — for example pull-downs or a watchdog on the enable lines — would remove the short interval of uncommanded driving that can occur if the control process itself dies. And the ultimate goal that framed the project throughout, taking a natural-language instruction and choosing a destination from it before handing off to the navigation policy, remains the capability toward which all of this builds.

13.4 Final Remarks

EAMS set out to turn low-cost, widely available hardware into a vehicle that could sense, decide and move safely, and to do so with a small team learning much of the field as it built. What it produced is a coherent, well-instrumented platform on which a simulation-trained policy drives real motors through a safety architecture that holds regardless of who is driving, together with an honest and specific account of what remains to reach full autonomy. For the team, the lasting value of the placement is less any single subsystem than the experience of building an integrated system across disciplines, under real hardware constraints, and reporting truthfully what it does and does not yet do.

A Project Timeline and Milestones

This appendix summarises the placement week by week, drawn from the team's daily and weekly logs and the project logbook. The collective Pearl-facing period ran from 22 June to 7 August 2026.

Table A.1: Weekly milestones over the placement.

Period	Focus	Outcome
Week 1 22–26 Jun	Onboarding and first ML-Agents steps	Toolchains installed (Unity, ML-Agents, Jetson SDK); first learning agents built (a cube trained to reach a target while avoiding walls)
Week 2 29 Jun–3 Jul	Simulated car and first navigation training	Differential-drive digital twin with ultrasonic and IR sensors; virtual sensors and friction calibrated; wall/obstacle-avoidance policy begun on the GPU machine
Week 3 6–10 Jul	Target-reaching policy and robustness; electronics review	Throttle/steer action space adopted; sensor suite fixed at 6 IR + 2 ultrasonic with the FL,FR,RL,RR,L,R order; randomised furniture; obstacles network trained toward ∼2.6M steps; power/motor-drive circuits simulated in Proteus and reviewed
Week 4 13–17 Jul	Hardware pivot and Raspberry Pi bring-up	Physical build begun; Pi control software, web app, safety layer and telemetry written; USB Ethernet link set up; car driven under power; Jetson OS installation started
Week 5 20–24 Jul	Jetson flashing and porting the control stack	Jetson Orin Nano flashed after host-setup difficulties; control stack ported from pigpio to Jetson.GPIO with the safety loop intact; shared repository created

Table A.1 – continued

Period	Focus	Outcome
Week 6 27–31 Jul	On-device policy and first motor-level autonomy	ONNX policy runner and Autonomy Lab built; motors running under code after the PWM/pin faults were resolved; dummy sensor/motor modes and interlocks added; camera feed added; service installed; demonstration during the Minister of ICT visit
Week 7 3–7 Aug	Consolidation, IMU and handover	MPU6050 brought up on I^2C bus 7 and the stall guard re-enabled; repositories consolidated; the 11-observation transfer-test agent added; documentation completed for handover

B Team and Working Model

This appendix is deliberately a consolidated account of how the seven-member team worked, rather than an individual-contribution assessment. Contributions are described at the level of workstreams, without ranking or assigning ownership to individuals.

B.1 Project Team

Table B.1: EAMS project team.

No.	Name	University / Institution	Programme
1	Nakyeyune Trudy Hope	Kyambogo University	Bachelor of Science in Biomedical Engineering and Mechatronics
2	Akankwasa Roberto Sanchez	University of the West of England	Bachelor of Science in Robotics Enineering
3	Ampeire Jean Marie Vianney	ISBAT University	Bachelor of Science in Electronics and Communications Engineering
4	Kyalimpa Mathias	Kyambogo University	Bachelor of Science in Biomedical Engineering and Mechatronics
5	Wandwali Chrispus	Kyambogo University	Bachelor of Science Technology- Physics
6	Muliho Denis Wafula	Nakawa Vocational	National Diploma in Mechanical Engineering
7	Amabe Godwin Pario	Mbarara University of Science and Technology	Bachelor of Science in Software Engineering

B.2 Working Model

The team worked in a research-based, self-directed mode. Each day opened with a stand-up to report progress and blockers and closed with a logged wrap-up, and leadership of the week rotated around the team, so coordinating the work — deciding what to build next, which on a research project is itself engineering — was a shared responsibility. Outstanding work was tracked on a shared board, source code was kept in Git and hosted on GitHub, and completed work was presented for critique and sign-off at intervals. Design decisions and their reasoning were written into the configuration file beside the values they justify, so the reasoning is met where it matters.

B.3 Collaborative Workstreams

The project was divided into broad technical workstreams that ran in parallel and converged at integration. These are listed to show how the work was organised, not to attribute them to individuals.

- **Simulation and reinforcement learning** — the Unity digital twin, the observation and action design, reward shaping, curricula and PPO training, and policy export.
- **Hardware, power and electronics** — the chassis and drive, the power architecture, battery management, and circuit design and simulation in Proteus.
- **Sensing and state estimation** — sensor selection, interfacing and calibration, and the pose Extended Kalman Filter validated in MATLAB.
- **Embedded software and safety** — the threaded control and telemetry stack, the motor safety layer, the ONNX policy runner, and the hardware-abstraction layer.
- **Computer vision** — the edge person-detection prototype and the camera subsystem.
- **Integration, documentation and presentation** — board bring-up, phase-gate testing, the project logbook and guides, and the progress presentations.

C Selected Code and Configuration

This appendix reproduces short, representative excerpts that help the reader understand the implementation. The full source remains in the project repository; the pin map in particular is authoritative in `config.py`, and this excerpt is illustrative.

C.1 Safety Constants and Pin Map

The safety layer is tuned entirely through named constants in the configuration module, so that no limit is a magic number elsewhere in the code.

Listing C.1: Safety-layer constants and motor pins (Jetson BOARD numbering).

```
# ---- motor pins (Jetson.GPIO BOARD numbering) ----
ENA = 33 # left channel PWM ENB = 32 # right channel PWM
IN1 = 24; IN2 = 26 # left direction
IN3 = 22; IN4 = 18 # right direction (swapped order reverses this pair)
PWM_FREQ_HZ = 2000

# ---- safety layer (the software IS the thermal protection) ----
DUTY_CAP = 0.55 # hard clamp, applied LAST, unconditional
SLEW_FULL_SCALE_S = 0.200 # ramp to target over ~200 ms
ZERO_CROSS_COAST_S = 0.080 # coast before reversing direction
DEADMAN_S = 0.300 # no packet for 300 ms -> zero
STALL_TIMEOUT_S = 1.0 # duty commanded, no motion 1 s -> latch
STALL_GYRO_THRESHOLD_COUNTS = 400 # ~3 deg/s; UNVALIDATED against real IMU
MOTOR_LOOP_HZ = 50
STALL_GUARD_ENABLED = True # re-enabled once the MPU6050 was fitted (bus 7)

# ---- IMU ----
I2C_BUS = 7 # Orin Nano header pins 3/5 are /dev/i2c-7, NOT bus 1
IR_LABELS = ["FL", "FR", "RL", "RR", "L", "R"] # policy observation order
```

C.2 PPO Training Configuration

The deployed policy was trained with the following configuration (abridged). The obstacle curriculum advances on training *progress*, not reward, because achievable reward falls as obstacle density rises.

Listing C.2: Abridged ML-Agents PPO configuration and obstacle curriculum.

```
behaviors:
  DifferentialCarAgent:
    trainer_type: ppo
    hyperparameters:
      batch_size: 1024
      buffer_size: 10240
      learning_rate: 0.0003
      beta: 0.01
      epsilon: 0.2
      lambd: 0.95
      num_epoch: 3
      learning_rate_schedule: linear
    network_settings:
      normalize: true
      hidden_units: 128
      num_layers: 2
    reward_signals:
      extrinsic: { gamma: 0.99, strength: 1.0 }
    max_steps: 5000000
    time_horizon: 64

environment_parameters:
  obstacle_count: # advanced on measure: progress
    curriculum:
      - { name: Clear, value: 0.0, threshold: 0.05 }
      - { name: Light, value: 6.0, threshold: 0.20 }
      - { name: Sparse, value: 15.0, threshold: 0.38 }
      - { name: Moderate, value: 25.0, threshold: 0.56 }
      - { name: Dense, value: 35.0, threshold: 0.75 }
      - { name: Full, value: 43.0 }
```

C.3 PWM Initialisation Order (a safety-relevant fix)

On the Jetson, a PWM channel is only driven if the pin's setup call comes immediately before its PWM object is constructed. Setting up both enable pins first leaves all but the last stuck at a fixed level — and "stuck fully enabled" on an H-bridge enable line bypasses the duty cap.

Listing C.3: Pairing each PWM pin's setup with its construction so the ordering cannot be got wrong.

```
def set_PWM_range(self, pin, pwm_range):
    if pin not in self._pwm:
        # Setup must come IMMEDIATELY before PWM construction; otherwise only
        # the last channel drives, and "stuck enabled" bypasses DUTY_CAP.
        # initial=LOW is deliberate: LOW means "channel off", the safe state.
        self.GPIO.setup(pin, self.GPIO.OUT, initial=self.GPIO.LOW)
        pwm = self.GPIO.PWM(pin, self._pwm_freq.get(pin, config.PWM_FREQ_HZ))
        pwm.start(0)
        self._pwm[pin] = pwm
```

D Test Records

This appendix records the principal tests carried out during bring-up and integration. Status is reported honestly: several tests were verified with the wheels off the ground, and some subsystems were validated in simulation rather than on the vehicle.

Table D.1: Test records.

ID	Test	Expected result	Actual result / status
T01	Dry-run full stack	Server, client and policy run with GPIO stubbed	Passed on laptop before deployment
T02	Motor direction mapping	Stick/command directions match wheel motion	Passed wheels-off after the pin-map correction
T03	Sensor acquisition	Ultrasonic (alternating), IR mask and IMU read into telemetry	Passed; IMU online at 0x68 on I^2C bus 7
T04	Deadman timeout	Wheels stop within 300 ms of lost commands	Passed; fires on tab close / disconnect
T05	Stall latch and reset	Latches on commanded duty with no motion; clears on reset	Passed; trips on blocks as expected, RESET clears it
T06	Duty cap	Applied output never exceeds 55 %	Passed; observed biting on the commanded channel
T07	PWM output on header	PWM present on BOARD pins 15/33	Verified with the bench script, motor battery disconnected
T08	Policy inference on-device	ONNX policy runs at 20 Hz within budget	Passed; inference ∼0.18 ms mean, 2.36 ms max in dry-run
T09	Autonomy pipeline (dummy motor)	Full loop runs on real Jetson without wheels turning	Passed; pose advances, deadman fires on disconnect
T10	Integrated drive under power	Vehicle drives under manual control	Passed on the Raspberry Pi; Jetson floor driving pending full sensor bring-up

Table D.1 – continued

ID	Test	Expected result	Actual result / status
T11	EKF pose estimation	Position error bounded vs. dead reckoning	Validated in MATLAB (∼0.02–0.07 m bounded); not deployed on the vehicle
T12	Vision prototype	Person detection on the camera feed	Demonstrated; not characterised for accuracy or frame rate

E Project Photographs

This appendix collects photographs and screenshots of the vehicle, its electronics and the system in operation. Captions explain what each image shows.

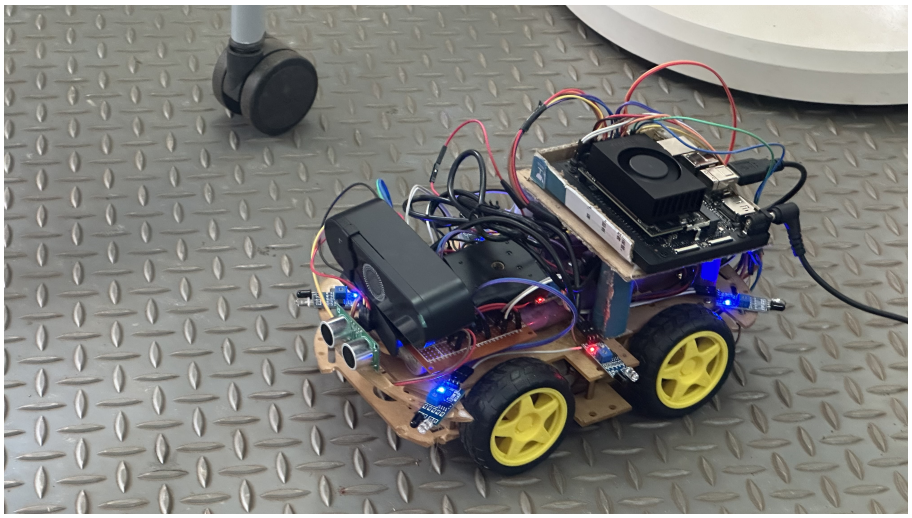


Figure E.1: The assembled EAMS differential-drive vehicle.

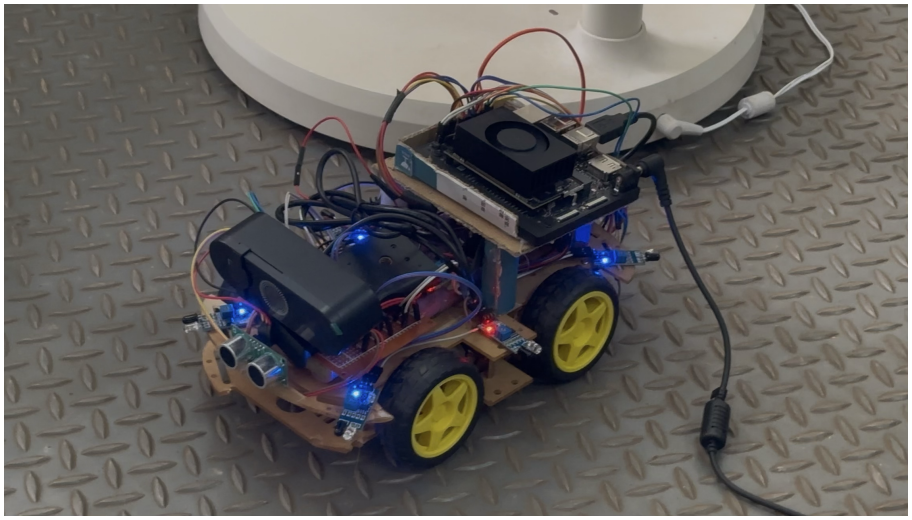


Figure E.2: The vehicle through the build: rewired on arrival and moved from breadboard to soldered stripboard, with the computing board changed from the Raspberry Pi to the Jetson Orin Nano.

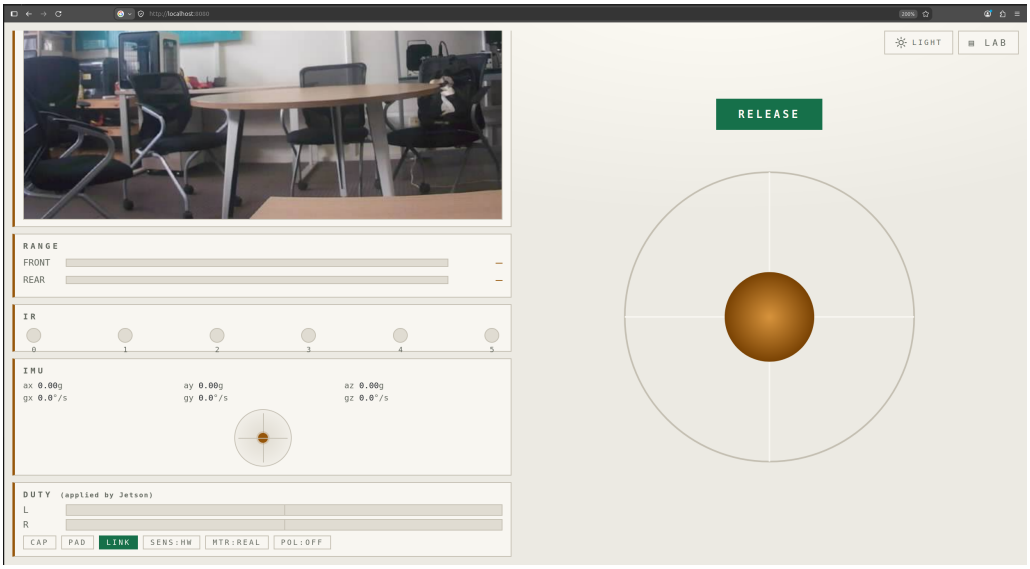


Figure E.3: The browser control interface during integration testing, showing live telemetry and the status of each safety guard.

Bibliography

Pearl AI Labs. Pearl AI labs. https://pearllabs.ug/, 2026. Official website; accessed August 2026.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.